

RESEARCH ARTICLE

MCKT: A Parallel Mamba-Based Model for Content-Aware Knowledge Tracing

YUMO YAO¹ AND HAI-TAO YU²¹Graduate School of Comprehensive Human Sciences, University of Tsukuba, Tsukuba, Ibaraki 305-0821, Japan²Institute of Library, Information and Media Science, University of Tsukuba, Tsukuba, Ibaraki 305-0821, Japan

Corresponding author: Hai-Tao Yu (yuhaitao@slis.tsukuba.ac.jp)

This work was supported in part by the Japan Science and Technology Agency (JST) Support for Pioneering Research Initiated by the Next Generation (SPRING) under Grant JPMJSP2124.

ABSTRACT Knowledge tracing (KT) aims to model students' evolving knowledge states and predict their performance on new exercises based on their past learning histories. Despite the recent advances, many existing KT models still rely primarily on explicit exercise attributes while overlooking contextual cues embedded in exercise content, such as implicit textual information and cognitive difficulty. This limitation hinders both their predictive performance and interpretability. Moreover, effectively capturing long-term dependencies in student learning trajectories remains challenging, especially in the presence of long or irregular interaction sequences. To address these challenges, we propose a parallel Mamba-based Content-aware Knowledge Tracing model (MCKT) that jointly models the semantic structure of exercise texts and cognitive difficulty of exercises. Specifically, MCKT incorporates a pre-trained language model to extract fine-grained semantic features from exercises. It employs two parallel Mamba-based sequence encoders for student responses and perceived difficulty, thereby capturing the temporal dynamics of knowledge states and difficulty perceptions. Furthermore, to address the challenge of students with sparse learning records, we introduce an adaptive enhancement module that more effectively captures short-sequence patterns. Leveraging Mamba's capability to efficiently capture long-range dependencies in student learning trajectories, MCKT achieves consistent gains in predictive performance and robustness, as confirmed by extensive experiments on multiple benchmark datasets. MCKT improves interpretability by mapping learning behaviors to meaningful textual and cognitive aspects of exercises, thereby enabling more transparent and adaptive educational support.

INDEX TERMS Content-aware modeling, educational data mining, intelligent tutoring systems, knowledge tracing.

I. INTRODUCTION

In recent years, technology-enhanced learning (TEL) has emerged as a key enabler of flexible and scalable education by breaking the temporal and spatial limitations of traditional in-person instruction [1]. TEL encompasses tools such as Massive Open Online Courses (MOOCs) [2] and Intelligent Tutoring Systems (ITS) [3], providing flexible and equitable learning opportunities. MOOCs offer widespread access to high-quality educational resources, while ITS delivers personalized and adaptive learning experiences. Meanwhile, these technologies enhance the educational landscape,

making it more accessible and tailored to individual learners' needs [4].

In intelligent tutoring systems, knowledge tracing (KT) is fundamental to diagnosing learners' capabilities and limitations, which in turn facilitates the delivery of personalized and adaptive learning experiences [5]. As a fundamental task in educational data mining, it models and predicts a student's knowledge state over time based on their interactions with educational exercises [6]. Effective knowledge tracing models are essential for guiding the provision of appropriate learning materials, ensuring that instruction is customized for each student's individual learning requirements [7]. To meet these needs, researchers have developed a variety of modeling approaches over time. Early knowledge tracing

The associate editor coordinating the review of this manuscript and approving it for publication was Senthil Kumar.

researchers adopted Hidden Markov Models, represented by Bayesian Knowledge Tracing [8], to monitor and reflect the evolution of students' knowledge states over time. With the advancement of deep learning techniques, Deep Knowledge Tracing [9] incorporated Long Short-Term Memory (LSTM) networks [10] into the knowledge tracing framework, demonstrating notable performance gains owing to their capacity to capture long-range dependencies among learning interactions. The rise of big data in education, with vast amounts of interaction data becoming available, has necessitated more sophisticated approaches. As a result, numerous deep learning-based approaches to knowledge tracing have been developed, showing considerable potential in modeling student learning behaviors [11], [12], [13], [14]. These models leverage sequence modeling techniques to provide more nuanced and accurate predictions of student performance, utilizing the sequential nature of student interactions to offer a more comprehensive understanding of the learning process.

Despite significant advancements in deep learning-based knowledge tracing, two key challenges remain unresolved. RNN-based knowledge tracing models [9], [11], including their variants such as LSTM and GRU, face limitations in modeling long-term dependencies, primarily due to the vanishing gradient issue [15]. Meanwhile, Transformer-based knowledge tracing models [12], [13], though highly effective, incur quadratic computational costs when processing long sequences [16], limiting their scalability for large-scale datasets. In addition to these architectural limitations, most existing models primarily rely on exercise IDs and knowledge components, neglecting the crucial role of exercise difficulty. Drawing inspiration from Item Response Theory [17] and the Rasch Model [18], we posit that a student's response is shaped not only by prior learning interactions but also by the difficulty level of upcoming exercises. However, prior work lacks explicit modeling of this factor, leading to suboptimal predictions.

Furthermore, another major limitation is the underutilization of rich textual information embedded in exercise content. Studies have pointed out that most knowledge tracing models primarily focus on exercise IDs while overlooking valuable semantic features present in side information [19], [20]. This omission constrains the model's capacity to comprehensively capture the complexity of student learning behaviors and exercise semantics, leading to information loss and suboptimal tracking of learning progress [21]. As illustrated in Fig. 1, e_1 and e_3 correspond to different exercises that assess distinct knowledge concepts. However, an analysis of their content reveals that the cognitive skills required for solving them exhibit underlying commonalities. Similarly, while e_4 and e_5 are both associated with the knowledge concept kc_4 (Area), solving e_5 inherently relies on an understanding of kc_1 (Squares). This highlights the deeper interdependencies among knowledge concepts in practical problem-solving. Considering the importance of textual information in faithfully modeling a student's knowledge

state, these challenges underscore the need for models that effectively integrate semantic content while efficiently handling long-term dependencies.

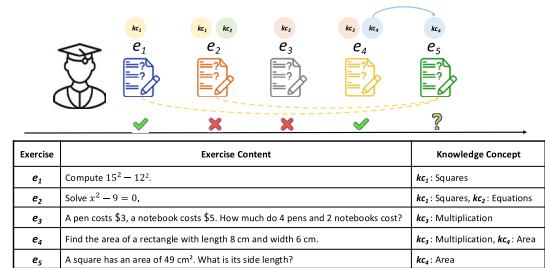


FIGURE 1. An illustration of the knowledge tracing process.

In response to these challenges, we present MCKT, a novel parallel Mamba-based knowledge tracing model that integrates semantic representations from exercise content and models learning behavior through state space dynamics. MCKT adopts a parallel-sequence modeling approach, where one Mamba module captures the student response sequence, while another models the exercise difficulty sequence, enabling it to learn both retrospective learning patterns and prospective challenges. To provide personalized difficulty signals, MCKT includes a Difficulty Evaluator that estimates the perceived difficulty of each exercise based on the student's proficiency, generating student-conditioned difficulty embeddings for downstream modeling. An effective integration of these two perspectives is achieved through a Mixing Layer, which fuses the outputs of both Mamba modules, ensuring that predictions incorporate real-time difficulty-adjusted learning states alongside sequential interaction patterns. Additionally, to incorporate the rich textual content of exercises, MCKT employs a pre-trained text encoder that transforms exercise content into dense vector representations. These embeddings are then combined with traditional features, such as exercise IDs and knowledge components, providing a more comprehensive representation of student interactions. While Mamba is well-suited for modeling long learning sequences, students with limited engagement history may suffer from long-tail generalization issues. To mitigate this, we introduce an Adaptive GRU module that enhances short-sequence representations, improving prediction accuracy for students with sparse learning data. By integrating these innovations, MCKT provides a more comprehensive and adaptive approach to knowledge tracing, capturing both temporal learning dynamics and the contextual difficulty of educational exercises. To summarize, MCKT introduces three major contributions:

- We introduce MCKT, a novel knowledge tracing model featuring parallel Mamba networks, which separately encode student responses and exercise difficulty signals, enabling more effective modeling of learning dynamics.
- We integrate content-aware embeddings with a pre-trained text encoder for exercise features while modeling difficulty. To address Mamba's short-sequence

limitations, we design an Adaptive GRU to enhance sparse learning representations and improve prediction accuracy.

- We demonstrate the effectiveness of MCKT in both traditional and context-enhanced knowledge tracing scenarios through extensive experiments on diverse datasets, including exercise content-unaware datasets (ASSIST09, ASSIST17, EdNet) and content-aware datasets (XES3G5M, Junyi, POJ).

This paper is organized as follows. Section II reviews the relevant literature. Section III outlines the preliminary concepts. The proposed framework is detailed in Section IV. Section V presents the experimental results and corresponding analysis. Finally, Section VI concludes the paper and discusses potential directions for future work.

II. RELATED WORK

This section presents an overview of three representative approaches to knowledge tracing, and provides a comprehensive review of the evolution of Mamba, along with its implementations in diverse application areas.

A. KNOWLEDGE TRACING

Knowledge tracing has gained significant attention in educational research for its potential to enhance intelligent educational services. Traditional approaches, such as Bayesian Knowledge Tracing (BKT) [8], estimate students' knowledge states using general parameters. Logistic models extend this by predicting the probability of correct responses based on student and knowledge concept information. Knowledge Tracing Machines (KTM) [22] further improve predictive accuracy and interpretability by employing factorization machines to capture interactions between features like student abilities and exercise difficulties.

The advent of deep learning has introduced new methods to model sequential student interactions. In light of our research focus, we categorize representative deep learning-based knowledge tracing methods into three groups: deep sequential models, attention-based models, and exercise content-based models.

1) DEEP SEQUENTIAL MODELS

Deep sequential models employ recurrent architectures to capture the temporal dynamics of student learning behavior. Deep Knowledge Tracing (DKT) [9] was the pioneering work that introduced LSTM networks into knowledge tracing, aiming to model the sequential progression of students' knowledge states. Dynamic Key-Value Memory Networks (DKVMN) [11] enhanced DKT by incorporating external memory, improving knowledge representation and tracking. Recent advancements, such as Question Difficulty Consistent KT (QDCKT) [23], combine LSTMs with additional neural networks to replace exercise IDs with difficulty levels, refining knowledge state estimations. Additionally, Difficulty Matching Knowledge Tracing (DIMKT) [24] proposes an

Adaptive Sequential Neural Network to explicitly model the relationship between students' knowledge states and exercises' difficulty levels during the learning process, enhancing interpretability and accuracy. Recent approaches like DyGKT [25] introduce dynamic graph learning to capture evolving relationships between students, questions, and concepts, addressing the growing scale of answering behaviors over time. Meanwhile, Q-MCKT [26] improves both the accuracy and interpretability of deep sequential models by utilizing a question-centric multi-experts contrastive learning framework, enhancing the representation of less interactive questions.

2) ATTENTION-BASED MODELS

Compared to earlier deep learning methods, attention mechanisms offer improved capability in modeling complex dependencies by enabling dynamic weighting of input elements [27]. Self-Attention Knowledge Tracing (SAKT) [12] first introduced self-attention into KT, allowing the model to selectively focus on relevant past interactions, while SAINT [13] further employed separate attention layers for exercises and responses to explicitly model student-exercise interactions. Deep Graph Memory Network (DGMN) [28] incorporates a forget gate and a dynamically evolving concept graph to better capture forgetting and concept relations, whereas IEKT [29] adopts a two-branch attention architecture to jointly estimate cognitive understanding and acquisition sensitivity, enhancing exercise-level interpretability. Subsequent refinements such as DTransformer [30] and SimpleKT [31] focus on stabilizing or simplifying self-attention for sequential KT, while other works specifically address issues like forgetting patterns [32], locality of attention [33], or length generalization [34]. More recently, extraKT [35] explicitly targets robust length extrapolation by adding linearly distance-decayed negative biases to attention scores, enabling models trained with short context windows to maintain strong performance on much longer interaction sequences.

While the aforementioned models primarily refine attention architectures, a parallel line of work enriches input representations to better capture learning context, particularly item difficulty. Context-aware attentive Knowledge Tracing (AKT) [36] introduces a psychometrics-inspired attention mechanism to model the interaction between student ability and item difficulty, and KTMFF [37] further fuses difficulty, exercise text, and student ability into a multi-head attention framework. DIMKT [24] extends this idea with a causal-aware attention module that aligns evolving student states with static item difficulties, and QDCKT [23] demonstrates in an LSTM-based setting that replacing exercise identifiers with difficulty-based encodings can substantially improve prediction quality. Nevertheless, most existing approaches still treat difficulty as a static attribute, which limits their ability to capture the dynamic

alignment between students' changing proficiency and the varying challenge posed by exercises over time.

3) EXERCISE CONTENT-AWARE MODELS

Exercise content-aware models explicitly integrate textual and contextual information into knowledge tracing, enhancing predictions by leveraging rich exercise content. Exercise-aware models, such as Exercise-aware Recurrent Neural Networks (EERNN) [38] and Exercise-aware Knowledge Tracing (EKT) [39], incorporate textual information from exercises to improve predictions. Review-aware Knowledge Tracing (RKT) [40] also contributes to this category by leveraging a relation-aware self-attention mechanism to model content relevance. AdaptKT [41] addresses domain-specific challenges by aligning textual features across domains through instance selection, distribution discrepancy minimization, and output fine-tuning. Open-ended Knowledge Tracing (OKT) [42] introduces the novel task centered on forecasting students' detailed responses to open-ended formats by combining program synthesis methods with knowledge tracing techniques.

Further, RAKT and its extension QRAKT [43] refine exercise-knowledge relationships by integrating interaction information, such as exercise contexts and time intervals, with student behaviors like guessing and slipping. Recent advances in large language models (LLMs) have further enhanced content-aware KT. The SINKT model [44] integrates exercise-level textual inputs into a graph-enhanced representation space, aligning LLM-encoded exercise content with topological knowledge structures for robust generalization. Agent4Edu [45] simulates student response behaviors using agent-based modeling, allowing for scalable pretraining of KT models, even in data-sparse scenarios. To address the cold-start problem, Guo et al. [46] propose an attribute-aware approach using LLMs to create semantically rich initial representations for zero-shot knowledge tracing.

B. ADVANCEMENTS AND APPLICATIONS OF MAMBA

The Mamba architecture constitutes a notable advancement in addressing the inherent limitations of attention-based approaches in time-series forecasting. While Transformer models [27] are proficient in capturing long-range dependencies, their effectiveness and robustness often decline when dealing with extended input sequences, mainly due to their quadratic time complexity $\mathcal{O}(N^2)$, which leads to considerable computational and memory overhead. Mamba, by contrast, offers a lightweight and efficient structure that enhances the capture of global dependencies and extracts temporal relationships more effectively [47]. These features make it a robust and scalable solution for handling sequential data, overcoming several critical challenges encountered by Transformer models.

The development of Mamba is grounded in the advancements of State Space Models (SSMs), which have provided

a structured and efficient approach to sequence modeling. S4 [48] pioneered the use of SSMs to capture long-range dependencies with linear complexity, and S5 [49] refined this with improved scalability and reduced computational costs. H3 [50] extended these capabilities, delivering competitive performance to Transformers in language modeling tasks. Mamba [51], building on these foundations, introduced an input-adaptive mechanism and hardware-efficient design, significantly improving inference speed and throughput while maintaining strong sequence modeling capabilities.

Mamba has found applications across diverse domains due to its efficient sequence modeling capabilities. In natural language processing, it has been utilized for language modeling and sequence prediction tasks with notable success [51]. In recommendation systems, Mamba4Rec [52] employs its efficiency for sequential recommendations, achieving competitive performance with reduced computational costs. Vision Mamba [53] applies Mamba to visual tasks by processing image patches as sequences, demonstrating state-of-the-art performance in classification and segmentation tasks, including ImageNet [54] classification. In medical imaging, Mamba has shown effectiveness in segmentation tasks [55]. These applications highlight Mamba's practical utility and efficiency in tackling a variety of sequence modeling challenges.

In the field of knowledge tracing, Mamba4KT [56] introduces the Mamba architecture into student learning behavior modeling. By leveraging selective state-space models, Mamba4KT enhances the efficiency and interpretability of knowledge tracing while maintaining strong predictive performance. However, despite its advantages, Mamba4KT lacks explicit consideration of exercise difficulty, which is essential for accurately modeling student learning behaviors [23], [57], [58]. Additionally, while Mamba excels in handling long sequences, it struggles with short sequences due to its reliance on structured state-space formulations [59].

III. PRELIMINARY

In this section, we begin by formulating the knowledge tracing problem and defining the notations used throughout the paper. We then provide an overview of state space models, which underpin our proposed approach. A summary of the notations is provided in Table 1.

A. PROBLEM FORMULATION

In knowledge tracing, we denote the set of students as S and the set of exercises as E . A student's learning history is represented as an interaction sequence $I_n = \{(e_1, r_1), \dots, (e_t, r_t), \dots, (e_n, r_n)\}$, where n is the length of the interaction sequence. Given the exercise $e_t \in E$ at the t -th step, $r_t \in \{0, 1\}$ indicates whether the student's response is correct or not. When it is necessary to specify the student and the exercise associated with a response, we use the notation $r_t^{i,j}$, which denotes the response of the i -th student to the

TABLE 1. The key notations used throughout the paper.

Notation	Definition
S	The set of students
E	The set of exercises
I	The interaction sequence of a student
e_t	The exercise attempted by student s at time step t
r_t	The response made by student s at time step t
$h(\tau)$	The latent state at continuous time τ
$h'(\tau)$	The derivative of the latent state $h(\tau)$
A	The state transition matrix in the SSM
B	The learnable input-to-latent parameter matrix
C	The learnable latent-to-output parameter matrix
$\bar{A}$	The discretized form of A
$\bar{B}$	The discretized form of B computed from A and B
Δ	The time step for discretization
$\exp(\cdot)$	The exponential function
h_t	The latent state at discrete time step t

j -th exercise at time step t . When the context is clear, the superscript indices are omitted for brevity. Formally, given the interaction history I_t of a student up to time t and the next exercise e_{t+1} , the objective of knowledge tracing is to estimate the probability of a correct response at time step $t+1$, namely, $P(r_{t+1} = 1 \mid e_{t+1}, I_t)$.

B. STATE SPACE MODELS

State space models (SSMs) provide a foundational and principled framework for sequence modeling, and they serve as the core building block of recent selective SSM architectures such as Mamba [51]. An SSM describes the evolution of a latent state over continuous time through linear ordinary differential equations. Here, τ denotes continuous time, and we consider an input signal $x(\tau) \in \mathbb{R}^{d_x}$ and an output signal $y(\tau) \in \mathbb{R}^{d_y}$. SSMs introduce a latent state $h(\tau) \in \mathbb{R}^N$ that summarizes the history of the input and mediates the mapping from $x(\tau)$ to $y(\tau)$. A standard continuous-time formulation is

$$h'(\tau) = Ah(\tau) + Bx(\tau), \quad (1)$$

$$y(\tau) = Ch(\tau), \quad (2)$$

where $A \in \mathbb{R}^{N \times N}$ is the state transition matrix, and $B \in \mathbb{R}^{N \times d_x}$ and $C \in \mathbb{R}^{d_y \times N}$ are the input and output projection matrices, respectively. All of these matrices are learnable parameters.

To apply SSMs to discrete-time sequences $\{x_t\}_{t=1}^T$, the continuous system is typically discretized under the zero-order hold (ZOH) assumption, which treats the input as piecewise constant between sampling instants with a fixed step size $\Delta > 0$. Denoting $h_t := h(t\Delta)$ and $y_t := y(t\Delta)$, we obtain the discrete-time recurrence

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad (3)$$

$$y_t = \bar{C}h_t, \quad (4)$$

where $\bar{C} = C$ and $\bar{A}, \bar{B}$ are the corresponding discrete-time parameters. Under ZOH, they can be expressed in closed form

as

$$\bar{A} = \exp(\Delta A), \quad (5)$$

$$\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)(\Delta B). \quad (6)$$

where I denotes the identity matrix. This discrete formulation allows SSMs to operate on sequences indexed by discrete time steps, similar to recurrent neural networks, while retaining a principled continuous-time interpretation. It enables linear-time recursive computation and efficient convolutional implementations for long-sequence modeling [60].

Building on this discretized SSM formulation, the Structured State Space model (S4) [48] leverages HiPPO [61] to impose a structured parameterization on the state matrix A , enabling stable and efficient long-range dependency modeling through both recurrent and convolutional implementations. More recently, Mamba [51] extends this formulation by introducing selective state space modeling, in which part of the SSM parameters become input-dependent. Specifically, while the state transition matrix A remains structured and shared across timesteps, the discretization step size and projection matrices are dynamically generated from the current input x_t . The selective parameters are computed as

$$\Delta_t = \text{softplus}(W_\Delta x_t), \quad (7)$$

$$B_t = W_B x_t, \quad (8)$$

$$C_t = W_C x_t, \quad (9)$$

where W_Δ, W_B , and W_C are learnable projection matrices, and the $\text{softplus}(\cdot)$ function ensures $\Delta_t > 0$ for numerical stability.

Under this input-conditioned parameterization, the discretized parameters in Eqs. (5) and (6) vary at each timestep, with the constant step size Δ and projection matrix B replaced by their input-dependent counterparts Δ_t and B_t . Consequently, the resulting state update follows the same zero-order hold formulation but is dynamically modulated at each timestep. Because Δ_t, B_t , and C_t are all conditioned on x_t , the model adaptively modulates the state transition and output projection at each step, constituting the core of selectivity in Mamba while retaining the linear-time computational advantages of SSMs.

In MCKT, this selective SSM formulation is instantiated independently in the two parallel Mamba modules—one for student response sequences and one for exercise difficulty sequences. The state transition matrix A is shared across timesteps within each module, while each module maintains its own set of projection parameters W_Δ, W_B , and W_C , allowing specialized selective state transitions tailored to each input modality.

IV. THE PROPOSED MCKT

In this section, we introduce the proposed parallel Mamba-based Content-aware Knowledge Tracing model (MCKT), which is shown in Fig. 2. MCKT integrates textual content embeddings extracted from pre-trained language models and

fuses them with student interaction sequences to enhance knowledge tracing. To jointly model student responses and perceived exercise difficulty, we employ a parallel Mamba architecture. A Difficulty Evaluator estimates exercise difficulty conditioned on the student's proficiency, providing personalized difficulty perceived representations. These are processed by a parallel Mamba module equipped with selective state-space models. A chain of L serially connected blocks, each composed of Parallel Mamba, LayerNorm, and FFN, enables progressive refinement of student representations. Simultaneously, an adaptive GRU captures short-sequence behaviors. A mixing layer then integrates the three pathways to construct a comprehensive representation of student learning dynamics. Compared to attention-based models [12], [13], [36], MCKT achieves higher efficiency and lower memory consumption while maintaining strong predictive performance.

A. STUDENT INTERACTION EMBEDDING

To effectively capture the relationship between exercise characteristics and student responses, we encode each student's exercise sequence by integrating exercise features with their corresponding answers. This design ensures that the model jointly learns both the inherent properties of the exercises and the student's problem-solving behavior, contributing to a richer and more accurate modeling of students' learning trajectories. Specifically, the interaction embedding $\hat{\mathbf{e}}_k$ is constructed by integrating the exercise feature vector $\mathbf{e}_k$ with a binary indicator of the student's response r_k . If $r_k = 1$, the embedding is $[\mathbf{e}_k : 0]$; otherwise, it is $[0 : \mathbf{e}_k]$, enabling the model to distinguish between correctly and incorrectly answered exercises while preserving the structural information of the exercises themselves.

$$\hat{\mathbf{e}}_k = \begin{cases} [\mathbf{e}_k : 0], & \text{if } r_k = 1 \\ [0 : \mathbf{e}_k], & \text{otherwise} \end{cases} \quad (10)$$

Thus, a student's interaction embedding is represented as $\hat{\mathbf{I}}_n = \{\hat{\mathbf{e}}_1, \dots, \hat{\mathbf{e}}_n\}$. Organizing interaction embeddings in this way enables the model to jointly encode the knowledge concepts involved in each exercise and the student's proficiency, thereby enhancing its predictive accuracy in knowledge tracing.

B. EXERCISE CONTENT EMBEDDING

In practical knowledge tracing scenarios, each exercise is associated with its corresponding textual content, which can reveal implicit relationships among exercises that are otherwise unobservable through exercise IDs or knowledge concept IDs alone. To leverage this information, we propose a Content Embedder that extracts semantic representations from exercise texts and integrates them into the model as a complementary input branch. Let T_i denote the textual content of exercise $e_i \in E$, where $i = 1, 2, \dots, n$. Each text T_i is first tokenized into a sequence of subword tokens:

$$\mathbf{x}_i = \text{Tokenizer}(T_i) = [v_{i,1}, v_{i,2}, \dots, v_{i,L_i}] \quad (11)$$

Here, $\mathbf{x}_i$ is the tokenized sequence for the i -th exercise, consisting of L_i tokens, and each $v_{i,j}$ is a token from the tokenizer's vocabulary. The token sequence $\mathbf{x}_i$ is then transformed into the input format required by RoBERTa [62], including token indices and attention masks, and subsequently fed into the pre-trained RoBERTa-base model, which produces a sequence of contextualized hidden states at its final layer:

$$\mathbf{H}_i = \text{RoBERTa}(\mathbf{x}_i) = [\mathbf{h}_{i,1}, \mathbf{h}_{i,2}, \dots, \mathbf{h}_{i,L_i}] \quad (12)$$

where $\mathbf{h}_{i,j} \in \mathbb{R}^d$ denotes the contextual embedding of the j -th token in the i -th exercise.

We adopt the pre-trained RoBERTa-base model without additional fine-tuning to maintain computational efficiency and training stability. We also evaluate two fine-tuning strategies (DAPT/E2E) of the content encoder in Appendix A. Fine-tuning yields no consistent improvement in AUC across datasets (with a marginal gain of 0.03% only on Junyi), while incurring substantially higher training time and GPU memory consumption. Similar marginal differences are observed on both Chinese (XES3G5M, Junyi) and English (POJ) datasets. Considering the limited accuracy gain relative to the additional computational cost, we retain the frozen encoder setting in the main experiments.

To obtain a fixed-size embedding for each exercise, we apply mean pooling over the token-level hidden states:

$$\mathbf{e}_i^c = \frac{1}{L_i} \sum_{j=1}^{L_i} \mathbf{h}_{i,j} \quad (13)$$

where $\mathbf{e}_i^c \in \mathbb{R}^d$ is the final content embedding of the i -th exercise.

By following this procedure, we obtain a set of dense semantic vectors $\mathbf{e}_1^c, \dots, \mathbf{e}_n^c$ corresponding to each exercise in the dataset. For a given student interaction sequence of length N , we align, at each time step, the content embedding of the exercised item with the corresponding student response using the interaction encoding scheme described in Eq. 10, resulting in a sequence of content-aware interaction embeddings $\{\mathbf{e}_1^c, \mathbf{e}_2^c, \dots, \mathbf{e}_N^c\}$. This sequence forms one of the input branches to the Parallel Mamba architecture.

C. DIFFICULTY ASSESSMENT

To accurately assess the difficulty of each exercise, we design a Difficulty Evaluator, which computes a weighted exercise accuracy rather than relying on a simple correctness ratio from the dataset. This approach accounts for the varying proficiency levels of students by assigning different weights to their responses, ensuring that exercise difficulty is estimated more robustly.

We compute the student ability weight w_i as an estimate of individual proficiency, derived from historical accuracy in the training set. Specifically, let R^i denote the set of all response records of student i , and let $r_{i,j}^{i,j}$ denote the set

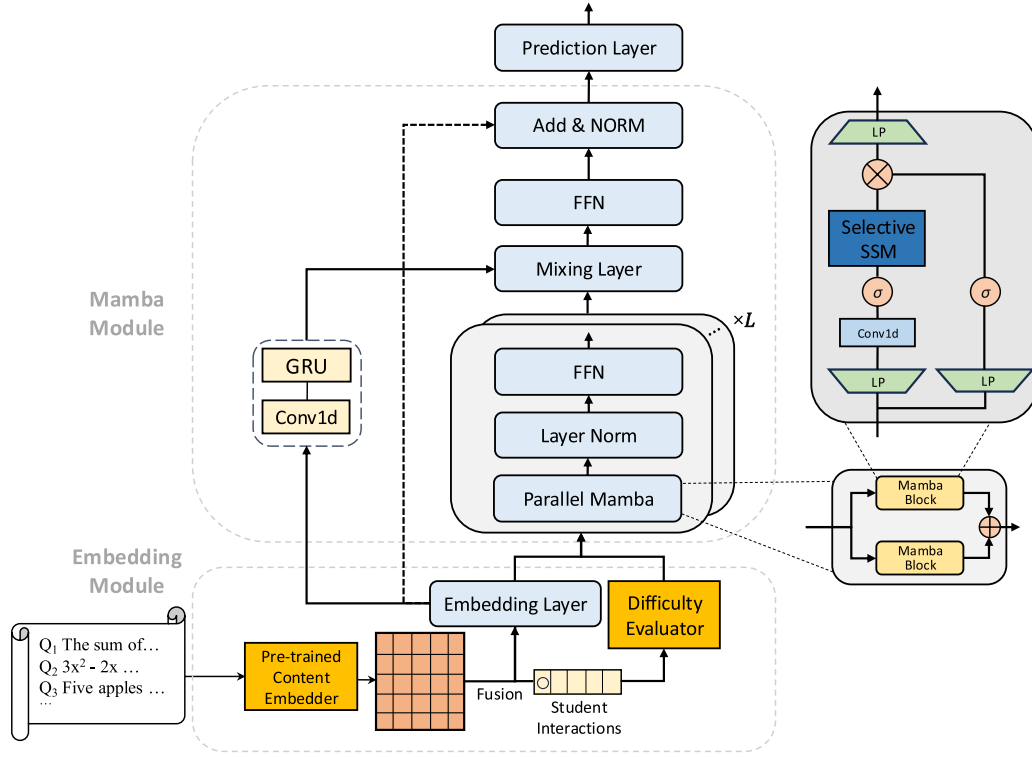


FIGURE 2. The framework of the proposed approach MCKT.

of correctness values corresponding to all responses from student i to exercise j . The proficiency weight is given by:

$$w_i = \frac{1}{|R^i|} \sum_j r_{i,j}^{i,j} \quad (14)$$

Given the proficiency weights w_i , we define the difficulty score $\beta_j \in [0, 1]$ for each exercise j , allowing students with higher ability to exert greater influence in the difficulty estimation. Let $R^{i,j}$ denote the set of response records where student i attempted exercise j , and let $R_+^{i,j} \subseteq R^{i,j}$ denote the subset of correct responses. To accommodate repeated attempts, the calculation is based on individual response records rather than aggregating by unique students.

When the number of responses for an exercise is small, the observed accuracy can be highly variable and may be skewed toward extreme values, potentially leading to unstable difficulty estimation. We therefore use a minimum number of responses threshold of 50 as a practical heuristic to balance statistical reliability. If the number of responses for exercise j is fewer than 50, we assign a default difficulty score of 0.5. The value 0.5 corresponds to a neutral, medium-level difficulty assumption, indicating that in the absence of sufficient evidence, the exercise is treated as neither clearly easy nor clearly difficult. The impact of varying this threshold and alternative smoothing strategies is further examined in the sensitivity analysis (Section V-F). The difficulty is computed

as:

$$\beta_j = \begin{cases} 1 - \frac{\sum_i w_i \cdot |R_+^{i,j}|}{\sum_i w_i \cdot |R^{i,j}|}, & \text{if } \sum_i |R^{i,j}| \geq 50 \\ 0.5, & \text{otherwise} \end{cases} \quad (15)$$

The normalization by $\sum_i w_i |R^{i,j}|$ ensures that the weighted accuracy term lies in $[0, 1]$, and therefore $\beta_j \in [0, 1]$. This formulation ensures that responses from more capable students contribute more heavily to the estimated difficulty. Furthermore, by operating at the response-record level, it naturally accounts for multiple attempts, better reflecting the iterative nature of learning.

To transform the scalar difficulty score into a vector representation suitable for downstream modeling, we combine it with the corresponding content embedding $\mathbf{e}_j^c$ and apply a linear projection:

$$\mathbf{e}_j^d = W_d[\mathbf{e}_j^c; \beta_j] + b_d \quad (16)$$

where $[\cdot; \cdot]$ denotes vector concatenation, $W_d \in \mathbb{R}^{d \times (d+1)}$ and $b_d \in \mathbb{R}^d$ are learnable parameters, and $\mathbf{e}_j^d \in \mathbb{R}^d$ is the resulting difficulty embedding of exercise j . This allows the difficulty representation to encode both semantic and statistical information.

For a student interaction sequence of length N , we similarly obtain, at each time step, the difficulty embedding of the exercised item, yielding a sequence of difficulty embeddings

$\{\mathbf{e}_1^d, \mathbf{e}_2^d, \dots, \mathbf{e}_N^d\}$. This sequence serves as the second input branch to the Parallel Mamba architecture.

It is worth noting that the proposed Difficulty Assessment is intended as a practical and lightweight proxy rather than a fully calibrated statistical model. It is most suitable for scenarios where exercises have sufficient historical response records and the student proficiency distribution remains relatively stable within the training set. Since both the student ability weights w_i and the exercise difficulty scores β_j are globally estimated from historical responses, the current design does not explicitly model temporal changes in student ability, distribution shift across learner groups, or dynamic variations in exercise difficulty. In addition, for cold-start or extremely sparse exercises, the default fallback strategy ($\beta_j = 0.5$) may still provide only a coarse approximation. Exploring online or dynamically updated difficulty estimation remains an important direction for future work.

D. PARALLEL MAMBA ARCHITECTURE

To effectively model both student interactions and exercise difficulties, we employ a parallel Mamba architecture consisting of two Mamba blocks operating in tandem. One block processes the content-aware interaction sequence $x^r = \{\mathbf{e}_1^c, \mathbf{e}_2^c, \dots, \mathbf{e}_N^c\}$, while the other encodes the corresponding difficulty sequence $x^d = \{\mathbf{e}_1^d, \mathbf{e}_2^d, \dots, \mathbf{e}_N^d\}$. This parallel-stream architecture enables the model to jointly capture behavioral patterns and exercise difficulty characteristics. The rationale for independent modeling lies in the fundamentally distinct nature of the two input streams. Response sequences capture student-side behavioral dynamics reflecting evolving knowledge states, whereas difficulty sequences encode exercise-side attributes representing the intrinsic challenge level of exercises.

Grounded in Item Response Theory, these two factors are independent latent variables that occupy different regions of the feature space, and conflating them within a single Mamba module would hinder the model's ability to disentangle and leverage each source effectively.

Each input sequence is first projected into a hidden representation via a linear transformation, followed by a convolutional projection and a gating mechanism. Specifically, for the interaction branch, the computation proceeds as follows:

$$\tilde{x}^r = x^r W_1^r + b_1^r, \quad (17)$$

$$x_{\text{conv}}^r = \text{SiLU}(\text{Conv1D}(\tilde{x}^r)), \quad (18)$$

$$z^r = \text{SiLU}(\tilde{x}^r W_2^r + b_2^r), \quad (19)$$

$$y^r = \left(\sum_{i=1}^N A^r(i) \cdot (x_{\text{conv}}^r \odot z^r)_{t-i} \right) W_3^r + b_3^r. \quad (20)$$

The same computation is applied to the difficulty branch:

$$\tilde{x}^d = x^d W_1^d + b_1^d, \quad (21)$$

$$x_{\text{conv}}^d = \text{SiLU}(\text{Conv1D}(\tilde{x}^d)), \quad (22)$$

$$z^d = \text{SiLU}(\tilde{x}^d W_2^d + b_2^d), \quad (23)$$

$$y^d = \left(\sum_{i=1}^N A^d(i) \cdot (x_{\text{conv}}^d \odot z^d)_{t-i} \right) W_3^d + b_3^d. \quad (24)$$

Here, all projection weights and biases are separately defined for each branch. Specifically, each branch has its own set of linear parameters $\{W_\ell^{(\cdot)}, b_\ell^{(\cdot)}\}_{\ell=1}^3$, where $(\cdot) \in \{r, d\}$ denotes the interaction and difficulty pathways, respectively. This parameter isolation allows the two Mamba blocks to specialize in modeling student behavior and exercise difficulty. The SSM kernels $A^r(i)$ and $A^d(i)$ are also independently learned for each stream.

E. INTERACTION DIFFICULTY FUSION

After processing the student interaction and difficulty sequences with the two parallel Mamba blocks, we obtain two sequences of hidden states $\{y_k^r\}_{k=1}^N$ and $\{y_k^d\}_{k=1}^N$, where $y_k^r, y_k^d \in \mathbb{R}^d$ denote the outputs at timestep k from the interaction and difficulty branches, respectively. For notational convenience, we denote $h_k^s := y_k^r$ and $h_k^d := y_k^d$. To explore effective fusion strategies, we consider three approaches to integrate the latent representations h_k^s and h_k^d at each timestep k .

The first approach is element-wise addition, which assumes both branches contribute equally to the fused representation:

$$f_k = h_k^s + h_k^d \quad (25)$$

The second strategy is early fusion, where we concatenate the two vectors and apply a feed-forward transformation with nonlinearity:

$$f_k = \phi \left([h_k^s; h_k^d] W_f + b_f \right) \quad (26)$$

The third method is a gated fusion mechanism, which dynamically balances the contribution of each branch through a learnable gate:

$$f_k = g_k \odot h_k^s + (1 - g_k) \odot h_k^d \quad (27)$$

where the gating vector $g_k \in \mathbb{R}^d$ is computed as:

$$g_k = \sigma \left([h_k^s; h_k^d] W_g + b_g \right) \quad (28)$$

Here, $W_f, W_g \in \mathbb{R}^{2d \times d}$, $b_f, b_g \in \mathbb{R}^d$ are learnable parameters, $\phi(\cdot)$ denotes an activation function such as ReLU, and $\sigma(\cdot)$ is the sigmoid function. These fusion strategies enable the model to capture complementary information from both behavioral and difficulty branches, yielding a comprehensive representation f_k for downstream prediction. Following a comparative analysis of the three fusion strategies, we adopt the **gated fusion mechanism** in the final model due to its consistently superior performance across all datasets. The detailed results of this analysis are presented in Section V-E.

F. ADAPTIVE GRU

While Mamba excels at capturing long-range dependencies, it may struggle with short sequences, which are common in knowledge tracing scenarios where students have limited

interaction histories. This is primarily because state-space models tend to smooth sequential information and are less responsive to rapid transitions in short sequences. To address this limitation, we introduce an *Adaptive GRU* module that dynamically adjusts its update behavior and selectively emphasizes fine-grained local patterns in short interaction sequences.

Let $\tilde{x}_k \in \mathbb{R}^d$ denote the input to the GRU at timestep k , and let the current sequence length be N . We define the hidden state update as:

$$h_k = (1 - \gamma_k) \cdot h_{k-1} + \gamma_k \cdot \tilde{h}_k \quad (29)$$

where the candidate hidden state $\tilde{h}_k$ is computed by:

$$\tilde{h}_k = \tanh(W_h \tilde{x}_k + U_h h_{k-1}) \quad (30)$$

To improve the GRU's adaptability to varying sequence lengths, we incorporate a length-aware gating mechanism in the computation of the update gate γ_k . Specifically, the update gate is conditioned not only on the current input and previous hidden state, but also on a sequence-level global representation $\bar{x} \in \mathbb{R}^d$, defined as the average input over the sequence:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N \tilde{x}_i \quad (31)$$

The length-aware update gate is then computed as:

$$\gamma_k = \sigma(W_\gamma \tilde{x}_k + U_\gamma h_{k-1} + V_\gamma \bar{x} + b_\gamma) \quad (32)$$

Here, $W_h, U_h, W_\gamma, U_\gamma, V_\gamma \in \mathbb{R}^{d \times d}$, $b_\gamma \in \mathbb{R}^d$ are learnable parameters. $\tanh(\cdot)$ and $\sigma(\cdot)$ denote the hyperbolic tangent and sigmoid activation functions, respectively.

By integrating the sequence-level global average $\bar{x}$ through the term $V_\gamma \bar{x}$ in the computation of the length-aware update gate, the GRU can dynamically adjust its sensitivity based on sequence length and content variance. This enables stronger adaptation in scenarios where students have only a few interactions, and local fluctuations carry more informative signals. The role of the length-aware update gate is examined from two perspectives: an ablation study in Section V-E isolates its structural contribution by removing the sequence-level conditioning, while a sequence-length-based analysis in Section V-G further explores how it influences model behavior across varying interaction histories. The resulting hidden states h_k are subsequently fused with the Mamba-based representation $\tilde{f}_k$ in the dynamic mixing layer.

G. DYNAMIC MIXING LAYER

To integrate long-term and short-term representations, we introduce a dynamic mixing layer that fuses the output of the Mamba fusion module and the Adaptive GRU at each timestep to produce an adaptive representation.

The final representation $\tilde{f}_k \in \mathbb{R}^d$ is computed as a weighted sum of the two sources:

$$\tilde{f}_k = \alpha_k \cdot \tilde{f}_k + (1 - \alpha_k) \cdot h_k \quad (33)$$

where $\alpha_k \in [0, 1]^d$ is a learnable gating vector that controls the relative contribution of the Mamba output and the GRU output at each timestep. To compute this gate, we use a learnable gating mechanism over the concatenated vector:

$$\alpha_k = \sigma(W_m[f_k; h_k] + b_m) \quad (34)$$

where $W_m \in \mathbb{R}^{2d \times d}$ and $b_m \in \mathbb{R}^d$ are learnable parameters, and $\sigma(\cdot)$ denotes the sigmoid function. This design allows the model to dynamically modulate its focus between long-term and short-term information based on the nature of the input at each timestep. As a result, it can more effectively handle sequences of varying lengths and interaction complexities.

H. REPRESENTATION REFINEMENT

After obtaining the fused representations $\tilde{f}_k$ from the mixing layer, we refine them through a Feed Forward Network (FFN), followed by a residual connection and layer normalization. This step enhances the expressiveness and stability of the learned representations. The FFN consists of two linear transformations with ReLU activation and dropout in between. Let the intermediate computations be:

$$z_k = W_1 \tilde{f}_k + b_1 \quad (35)$$

$$u_k = W_2 \cdot \text{Dropout}(\text{ReLU}(z_k)) + b_2 \quad (36)$$

We then apply a residual connection to obtain:

$$r_k = \tilde{f}_k + u_k \quad (37)$$

Finally, layer normalization is applied to stabilize the output:

$$h_k = \text{LayerNorm}(r_k) \quad (38)$$

where $h_k \in \mathbb{R}^d$ denotes the final refined representation at timestep k , and will be used as input to the prediction layer.

This design ensures that the fused features are nonlinearly transformed, regularized, and normalized, resulting in stable and expressive representations for downstream prediction tasks.

I. PREDICTION LAYER

Before the final prediction, we further compute a sequence-level summary over the refined representations h_k using learned summary attention weights. Specifically, a scalar score is assigned to each timestep and normalized over the sequence:

$$e_k = \mathbf{w}_a^\top h_k + b_a, \quad (39)$$

$$\beta_k = \frac{\exp(e_k)}{\sum_{j=1}^T \exp(e_j)}, \quad (40)$$

where β_k denotes the learned summary attention weight at timestep k , reflecting the relative contribution of the corresponding historical interaction in the final prediction stage. Based on these weights, a global summary vector is computed as

$$s = \sum_{k=1}^T \beta_k h_k. \quad (41)$$

To adaptively inject this global summary back into each timestep, we further define

$$\tilde{h}_k = h_k + \sigma(W_s h_k + b_s) \odot s, \quad (42)$$

where $W_s \in \mathbb{R}^{d \times d}$ and $b_s \in \mathbb{R}^d$ are learnable parameters, and $\odot$ denotes element-wise multiplication.

Based on the refined representation $\tilde{h}_k$, the prediction layer estimates the probability $\hat{y}_k$ that the student will correctly respond to the next exercise. This is done by applying a linear transformation followed by a sigmoid activation:

$$z_k = W_o^T \tilde{h}_k + b_o, \quad (43)$$

$$\hat{y}_k = \sigma(z_k) = \frac{1}{1 + e^{-z_k}}, \quad (44)$$

where W_o and b_o denote the output weight matrix and bias term, respectively.

To optimize the model parameters, we employ the binary cross-entropy loss $\mathcal{L}_{\text{BCE}}$, which measures the divergence between the predicted outcome $\hat{y}_k$ and the actual student response r_{k+1} at the next time step:

$$\mathcal{L}_{\text{BCE}} = - \sum_{i=1}^n \delta_{k+1}^i (r_{k+1} \log \hat{y}_k + (1 - r_{k+1}) \log(1 - \hat{y}_k)) \quad (45)$$

where δ_{k+1}^i is an indicator function equal to 1 if e_i is the exercise attempted by the student at step $k + 1$, and 0 otherwise.

This completes the forward pass of MCKT, enabling it to produce accurate predictions by jointly modeling exercise content, student behavior, and exercise difficulty in a unified framework.

V. EXPERIMENTS

To empirically evaluate the proposed MCKT model, we conduct a series of experiments on diverse real-world datasets, aiming to address the following research questions and assess the model's performance in terms of both accuracy and efficiency.

- **RQ1:** How does MCKT perform across different datasets compared to state-of-the-art knowledge tracing models?
- **RQ2:** How does MCKT scale in computational efficiency, and how does its predictive performance change as the context window length increases?
- **RQ3:** What is the impact of Adaptive GRU on MCKT's performance, particularly in handling short sequences?
- **RQ4:** How sensitive is MCKT to key hyperparameters and design choices?

To answer these questions, we first outline the datasets used for evaluation, and then present the subsequent experimental configurations and assessment criteria, including evaluation metrics, baseline models, and training configurations. We then present an in-depth analysis of MCKT's performance, efficiency, and robustness through ablation and sensitivity studies.

TABLE 2. The statistics of content-unaware datasets.

Statistic	ASSIST09	ASSIST17	EdNet
Students	4,151	1,709	5,000
Exercises	16,891	3,162	10,795
KCs	101	102	188
Interactions	274,590	942,816	225,625
Subjects	Math	Math	Linguistics

A. DATASETS

To comprehensively evaluate our model, we divide the datasets into two groups depending on the availability of textual content. Specifically, we define a simplified version of our model, denoted as **MKT**, by removing the Content Embedder module from **MCKT**. This variant is designed for content-unaware datasets. Accordingly, we organize our experiments into two parts: (1) evaluating **MKT** on ASSIST09, ASSIST17, and EdNet; (2) evaluating **MCKT** on XES3G5M, Junyi, and POJ, compared with other exercise content-aware methods.

1) EXERCISE CONTENT-UNAWARE DATASETS

Exercise content-unaware datasets are agnostic to textual information of exercises, meaning that they do not include or leverage any exercise content in the modeling process. We assess the sequence modeling performance of our **MKT** model using three exercise content-unaware datasets that are widely adopted in prior work, as shown in Table 2.

ASSIST09¹ is collected from the ASSISTments platform during the 2009–2010 academic year [63]. This dataset is preprocessed by filtering interactions to retain only those involving exercises annotated with knowledge concepts and students who have attempted at least three questions.

ASSIST17² originates from the ASSISTments 2017 Data Mining Competition. For consistency, it is subjected to a preprocessing pipeline similar to that applied to ASSIST09.

EdNet³ is a large-scale dataset introduced by Choi et al. [64], comprising over 130 million interaction records from approximately 780,000 students. To improve computational efficiency, we randomly sample the learning histories of 5,000 students for evaluation.

The default maximum sequence length is set to 50 for fair comparison with prior KT models, with longer sequences split into multiple subsequences. We randomly select 80% of students for training and validation, and the remaining 20% for testing. Among the 80%, we further split 90% for training and 10% for validation.

¹<https://sites.google.com/site/assistmentsdata/home/2009-2010-assistment-data>

²<https://sites.google.com/view/assistmentsdatamining/data-mining-competition-2017>

³<https://github.com/rriid/ednet>

2) EXERCISE CONTENT-AWARE DATASETS

To further assess the impact of incorporating textual information, we evaluate our MCKT model on three exercise content-aware datasets, as presented in Table 3.

XES3G5M,⁴ introduced by Liu et al. [65], is a real-world online mathematics learning dataset enriched with diverse contextual features. It includes tree-structured relations among knowledge concepts, various exercise types, detailed textual content, and timestamp information. The dataset comprises 7,652 exercises, 865 knowledge concepts, and a total of 5,549,635 interactions from 18,066 students.

Junyi⁵ is derived from Junyi Academy (2015) [66] and contains student exercise records. We scraped the website to include textual content, focusing on 838 distinct exercises, filtering out non-textual ones.

POJ⁶ includes computer programming exercises from the Peking Online Judge platform [40]. The dataset contains over 26.67 million records from approximately 0.22 million students.

TABLE 3. The statistics of content-aware datasets.

Statistic	XES3G5M	Junyi	POJ
Students	18,066	5,000	22,916
Exercises	7,652	838	2,751
KCs	865	41	–
Interactions	5,549,635	8,162,247	996,240
Subjects	Math	Math	CS
Language	Chinese	Chinese	English

B. EXPERIMENTAL SETUP

In this section, we detail the experimental setup used to evaluate the performance of our model. This includes the baseline models, evaluation metrics, and implementation details.

1) BASELINE METHODS

The proposed model was compared with twelve representative baseline models, which were categorized into two groups based on whether they incorporated exercise content-aware features.

a: SEQUENTIAL MODELING METHODS

We compare our model with a range of representative sequential knowledge tracing methods. Early probabilistic models include **BKT** [8], which uses hidden Markov models to model student knowledge states. **KTM** [22] applies factorization machines to incorporate side information such as exercise and concept IDs. Deep learning models include **DKT** [9], which uses RNNs to capture temporal dynamics, and **DKVMN** [11], which introduces a memory-augmented

architecture. Attention-based methods such as **SAKT** [12], **AKT** [36], and **SAINT** [13] leverage self-attention to identify relevant past interactions. **IEKT** [29] introduces a two-branch structure to estimate cognition and acquisition sensitivity, while **DIMKT** [24] models personalized acquisition and exercise difficulty. **DTransformer** [30] applies stability constraints in Transformer-based KT modeling. Additionally, we include **simpleKT** [31], a lightweight yet competitive baseline, and **Mamba4KT** [56], which employs selective state-space Mamba blocks for efficient and interpretable prediction.

b: EXERCISE CONTENT-AWARE KNOWLEDGE TRACING METHODS

Several methods incorporate exercise textual content to enhance knowledge tracing performance. **EERNN** [38] combines textual features with student interaction sequences via RNNs. **EKT** [39] builds upon EERNN by modeling relations among latent knowledge concepts. **RKT** [40] introduces a relation-aware self-attention mechanism to integrate both contextual and performance signals. As the original implementation of **SINKT** [44] is unavailable, we replicate its core idea by extracting pre-trained textual embeddings and aligning them with graph-based representations to approximate the integration of content and structure. To systematically compare content-aware variants, we extend several baseline models with a shared *Content Embedder*. Specifically, we integrate semantic representations into **DKT** [9] (denoted as **DKT-cont**), **AKT** [36] (**AKT-cont**), and the lightweight **simpleKT** [31] (**SKT-cont**), enhancing each model's ability to capture textual context while preserving their original architectures. Lastly, we augment **Mamba4KT** [56] with content embeddings to construct **Mamba4KT-cont**, enabling us to evaluate Mamba's effectiveness in content-aware scenarios.

2) EVALUATION METRIC

To assess the effectiveness of our model and the baselines, we adopt two widely used evaluation metrics in knowledge tracing: Area Under the ROC Curve (AUC) and Accuracy (ACC). AUC reflects the model's capability to differentiate between correct and incorrect responses, while ACC indicates the proportion of predictions that match the actual student answers.

3) IMPLEMENTATION DETAILS

We implemented all baseline models and our proposed MCKT model using Python and the PyTorch library. To ensure consistency across experiments, we followed the same preprocessing procedures for all datasets. During training, we used the Adam optimizer with a learning rate of 0.001, a batch size of 128, and trained for 100 epochs. An early stopping mechanism with a patience of 10 epochs was employed based on the validation loss.

For the exercise content encoder, we used the pretrained RoBERTa-base model with 768-dimensional hidden states.

⁴<https://github.com/ai4ed/XES3G5M>

⁵<https://pslcdatashop.web.cmu.edu/DatasetInfo?datasetId=1275>

⁶<http://poj.org/>

Only the mean pooled output of the final hidden layer was used as the content embedding. To reduce computational overhead, RoBERTa parameters were frozen during training and not fine-tuned. For the Mamba modules, we followed the recommended configuration from Gu and Dao [51], employing selective state space layers with kernel size 3 and hidden dimension 256. Each Mamba block consists of a convolutional projection, a SiLU gating mechanism, and a selective SSM component. We used two parallel Mamba blocks in our parallel-stream architecture for modeling response and difficulty sequences, respectively.

All experiments were conducted in parallel using four NVIDIA A100-PCIE-40GB GPUs. We adopted PyTorch's DataParallel to enable multi-GPU training and ensure efficient computation across large datasets and model variants. To ensure reproducibility, we have made our code and datasets publicly available.⁷

C. PERFORMANCE COMPARISON (RQ1)

To comprehensively evaluate the effectiveness of MCKT, we conduct two sets of experiments. The first set focuses on datasets without exercise textual information, where we compare MKT (MCKT without content embedding) against baseline knowledge tracing models. This comparison isolates the contributions of our parallel-Mamba structure and Adaptive GRU while ensuring a fair evaluation against traditional deep knowledge tracing methods.

The second set of experiments evaluates MCKT on datasets that include exercise textual content, comparing it against the same baseline models. This setting highlights the benefits of incorporating exercise content-aware encoding and examines how textual information enhances knowledge tracing performance. In both experiments, we report AUC and ACC metrics and perform paired t-tests between our model and each baseline model to assess statistical significance, with an asterisk (*) denoting p-values below 0.05.

TABLE 4. The performance comparison between MCKT and baseline methods on three content-unaware datasets. * indicates statistical significance ($p < 0.05$).

Model	ASSIST09		ASSIST17		EdNet	
	AUC	ACC	AUC	ACC	AUC	ACC
BKT	0.7213	0.7235	0.6365	0.6542	0.6667	0.6414
KTM	0.7292	0.7235	0.6732	0.6606	0.6894	0.6504
DKT	0.7337	0.7019	0.7070	0.6790	0.6678	0.6316
DKVMN	0.6879	0.6868	0.6670	0.6655	0.6575	0.6417
SAKT	0.7498	0.7424	0.6648	0.6645	0.6345	0.6110
AKT	0.7561	0.7483	0.7034	0.6701	0.6853	0.6583
SAINT	0.6976	0.6977	0.6534	0.6476	0.6543	0.6402
IEKT	0.7672	0.7464	0.7354	0.6803	0.6732	0.6524
DIMKT	0.7634	0.7462	0.7203	0.6726	0.6752	0.6657
DTransformer	0.7656	0.7517	0.7434	0.6976	0.6841	0.6532
simpleKT	0.7681	0.7553	0.7421	0.6812	0.6813	0.6641
Mamba4KT	0.7545	0.7439	0.7441	0.6959	0.6792	0.6581
MKT	0.7778*	0.7634*	0.7561*	0.7161*	0.7002*	0.6741*

Table 4 presents the performance of MKT and baseline methods on three content-unaware knowledge tracing

datasets. MKT consistently achieves the best results across all datasets and metrics, significantly outperforming both classical models (e.g., BKT, KTM) and deep learning-based approaches (e.g., DKT, SAKT, SAINT). This performance gain highlights the effectiveness of MKT's parallel Mamba architecture in capturing complex temporal dependencies in student interaction sequences, as well as the contribution of the Adaptive GRU in addressing the long-tail user problem commonly seen in short sequences. Among attention-based models, AKT demonstrates strong performance on EdNet, while IEKT achieves competitive AUC scores on ASSIST09 and ASSIST17, reflecting their capacity to incorporate student ability and cognitive variability. However, MKT still consistently outperforms both AKT and IEKT across all benchmarks, demonstrating that the combination of Mamba-based temporal modeling and difficulty-aware adaptive mechanisms offers superior generalization.

Although the Mamba4KT, which uses a single Mamba block, shows competitive performance compared to recent baselines such as simpleKT and DTransformer, its improvements are less stable, particularly on ASSIST09 and EdNet. This suggests that while the Mamba module enhances sequential modeling, the full advantage is only realized when integrated with adaptive mechanisms and a more expressive design, as in MKT. The statistically significant improvements (denoted by *) further confirm the robustness and generalizability of MKT across datasets of varying scale and sparsity.

TABLE 5. The performance comparison between MCKT and baseline models on three content-aware datasets. * indicates statistical significance ($p < 0.05$).

Models	XES3G5M		Junyi		POJ	
	AUC	ACC	AUC	ACC	AUC	ACC
DKT	0.8177	0.8288	0.8014	0.7741	0.6325	0.6467
EERNN	0.8124	0.8212	0.8214	0.7836	0.6854	0.6732
EKT	0.8312	0.8311	0.8461	0.7908	0.6884	0.6778
RKT	0.8411	0.8351	0.8650	0.7828	0.7154	0.7032
SINKT	<u>0.8426</u>	<u>0.8367</u>	<u>0.8654</u>	<u>0.7954</u>	0.7173	0.7041
DKT-cont	0.8057	0.8211	0.8031	0.7682	0.6553	0.6421
AKT-cont	0.8326	0.8272	0.8452	0.7889	0.7153	0.7032
SKT-cont	0.8402	0.8353	0.8646	0.7914	<u>0.7364</u>	<u>0.7325</u>
Mamba4KT-cont	0.8315	0.8241	0.8561	0.7944	0.7251	0.7314
MCKT	0.8451*	0.8385*	0.8688*	0.7991*	0.7524*	0.7412*

Table 5 reports the performance of MCKT and baseline models on three exercise content-aware datasets. MCKT achieves the highest AUC and ACC scores across all benchmarks, demonstrating the effectiveness of integrating exercise text semantics and dual-stream temporal modeling. Compared to traditional models such as DKT and EKT, and content-aware baselines like RKT and EERNN, MCKT consistently delivers improved performance. While Mamba4KT-cont shows competitive results, particularly on POJ, its performance on Junyi is less stable, suggesting that long-range modeling alone may not suffice in short-sequence scenarios without complementary mechanisms.

⁷<https://github.com/ii-research/MCKT>

SINKT, which combines pre-trained text embeddings with structural alignment, performs strongly on XES3G5M and Junyi, indicating the value of jointly modeling semantic and relational information. Nevertheless, MCKT outperforms SINKT across all datasets, reflecting the benefit of its parallel temporal structure and difficulty-aware integration. AKT-cont and SKT-cont, both enhanced with our content encoder, also show strong performance, approaching or exceeding earlier content-aware baselines, and demonstrating that the semantic module generalizes well across different sequential architectures. In contrast, DKT-cont underperforms its vanilla counterpart on XES3G5M, suggesting that naive feature concatenation without structural integration may introduce noise. These results underscore the importance of tightly coupling content information with temporal modeling, as exemplified by the design of MCKT.

D. EFFICIENCY ANALYSIS (RQ2)

This section evaluates how MCKT scales in computational efficiency and how its predictive performance changes as the context window length increases. Specifically, we first examine training and inference costs across multiple datasets. Table 6 presents a comparative analysis of various knowledge tracing models, highlighting differences in training time (TT), inference time (IT), and GPU memory usage (GM). Additionally, Fig. 3 provides a detailed comparison of MCKT, SAINT, and Mamba4KT, illustrating how computational costs scale with increasing sequence length.

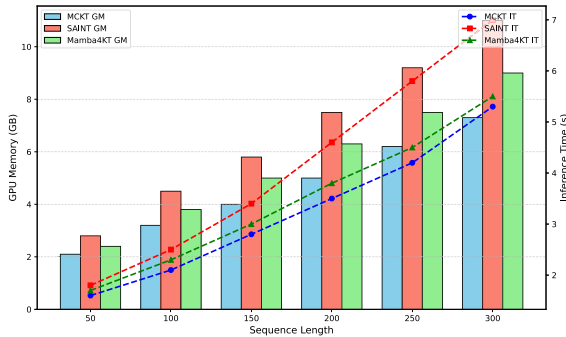


FIGURE 3. The comparison of inference memory and GPU memory across sequence lengths.

From Table 6, MCKT demonstrates lower computational costs across multiple datasets. In ASSIST09, MCKT achieves the shortest training time (240s) and the lowest memory consumption (1.7GB) while maintaining a minimal inference time (1.6s), outperforming existing models. On ASSIST17, although MCKT's total training time is slightly higher than that of SAKT, it maintains lower inference time and memory usage. For the larger-scale XES3G5M dataset, MCKT outperforms both Mamba4KT and SAINT in terms of inference time and memory efficiency. Compared to Transformer-based models such as SAINT and DTransformer, MCKT achieves

more stable computational efficiency, making it a suitable solution across various dataset scales.

Fig. 3 further validates the scalability of MCKT across different sequence lengths. As sequence length increases, SAINT exhibits a rapid increase in both GPU memory usage and inference time, whereas MCKT consistently maintains lower computational costs. Compared to Mamba4KT, MCKT achieves lower inference time in long-sequence scenarios, indicating that its state-space modeling strategy effectively mitigates the computational burden associated with long sequences.

In addition to computational efficiency, we further examine how predictive performance changes as the maximum context window length increases. We vary $T \in \{10, 25, 50, 100, 200\}$ and report AUC and ACC on ASSIST09, ASSIST17, and XES3G5M. For sequences exceeding T , chronological segmentation is applied consistently during training and evaluation.

As shown in Table 7, MCKT maintains stable predictive performance as the context window increases, with improvements gradually saturating beyond moderate lengths. These results indicate that MCKT can effectively utilize extended contextual information without performance degradation, consistent with the scalable design of selective state space models.

E. ABLATION STUDY (RQ3)

To assess the impact of key architectural components in MCKT, and to clarify what MCKT contributes beyond a direct application of Mamba to knowledge tracing, we conduct an ablation study targeting three core architectural decisions: (a) parallel dual-stream vs. single-stream design (*Single Mamba* variant); (b) the contribution of the explicit difficulty branch (*w/o Difficulty* variant); and (c) the choice of fusion mechanism (Table 9). Beyond these, we further examine the effectiveness of Adaptive GRU for short-sequence modeling and the role of content information. The full set of variants is as follows:

- **w/o Content:** Removes the content encoder, using only student interaction information as model input.
- **w/o Difficulty:** Removes the difficulty input branch, modeling only student response behavior.
- **w/o Length-aware Gate:** Removes the global representation term $V_{\gamma}\bar{x}$ from the update gate in Adaptive GRU, reverting it to the standard GRU without sequence-level conditioning.
- **w/o Adaptive GRU:** Removes the entire Adaptive GRU branch, using only the parallel Mamba representation for prediction.
- **w/o Student Weighting:** Sets all student ability weights $w_i = 1$, effectively disabling the weighting mechanism in difficulty estimation.
- **Parallel-RNN:** Replaces both Mamba modules with two independent RNNs, motivated by the structural similarity between SSMs and RNNs. This variant

TABLE 6. The comparison of computational efficiency across models and datasets.

Model	ASSIST09			ASSIST17			XES3G5M		
	TT (s)	IT (s)	GM (GB)	TT (s)	IT (s)	GM (GB)	TT (s)	IT (s)	GM (GB)
SAKT	246	1.8	1.9	134	1.7	0.9	487	2.0	1.1
SAINT	423	3.3	6.9	394	2.2	5.4	1043	4.5	6.3
DTransformer	1137	14.2	11.5	1135	7.0	7.2	2085	9.0	12.5
extraKT	782	9.2	13.2	512	6.0	11.4	1564	7.9	10.4
Mamba4KT	316	5.4	4.7	253	2.0	2.5	912	3.6	4.8
MCKT (Ours)	240	1.6	1.7	<u>140</u>	<u>1.8</u>	<u>1.0</u>	730	2.8	4.5

TABLE 7. The predictive performance of MCKT under varying maximum context window lengths.

T	ASSIST09		ASSIST17		XES3G5M	
	AUC	ACC	AUC	ACC	AUC	ACC
10	0.7721	0.7584	0.7489	0.7101	0.8398	0.8359
25	0.7745	0.7614	0.7514	0.7128	0.8425	0.8371
50	<u>0.7778</u>	<u>0.7634</u>	0.7561	0.7161	0.8451	0.8385
100	0.7787	0.7635	<u>0.7558</u>	<u>0.7158</u>	<u>0.8455</u>	<u>0.8387</u>
200	0.7774	0.7631	0.7551	0.7156	0.8458	0.8388

TABLE 8. The performance comparison between MCKT and its variant methods.

Models	XES3G5M		Junyi		POJ	
	AUC	ACC	AUC	ACC	AUC	ACC
w/o Content	0.8421	0.8372	0.8560	0.7961	0.7422	0.7350
w/o Difficulty	0.8435	0.8379	0.8471	0.7981	0.7430	0.7367
w/o Length-aware Gate	0.8405	0.8359	0.8512	0.7972	0.7496	0.7323
w/o Adaptive GRU	0.8393	0.8348	0.8501	0.7963	0.7486	0.7309
w/o Student Weighting	0.8437	0.8380	0.8523	0.7953	0.7487	0.7389
Parallel-RNN	0.8362	0.8354	0.8698	0.8003	0.7440	0.7380
Concat Fusion	0.8422	0.8365	0.8525	0.7949	0.7484	0.7401
Single Mamba	0.8433	0.8361	0.8445	0.7943	0.7418	0.7369
MCKT	0.8451	0.8385	0.8688	0.7991	0.7524	0.7412

helps assess whether RNNs are more suitable for short-sequence datasets like Junyi.

- **Concat Fusion:** Replaces the mixing layer with a simple feature concatenation, testing the importance of adaptive fusion.
- **Single Mamba:** Uses a single Mamba to jointly encode both response and difficulty information, instead of two separate modules.

The performance differences among model variants in Table 8 reflect the individual contributions of key components within MCKT. Removing the content encoder (*w/o Content*) results in a clear performance drop, indicating that semantic information from exercise texts plays an important role in enhancing representation quality. Similarly, removing the difficulty input branch (*w/o Difficulty*) degrades performance, suggesting that exercise difficulty provides complementary information beyond student response patterns.

Removing the entire Adaptive GRU branch (*w/o Adaptive GRU*) leads to consistent performance degradation across all three datasets, confirming that Adaptive GRU provides useful local sequential cues beyond the parallel Mamba

backbone. The larger drop on the shorter-sequence datasets, especially on *Junyi* relative to *XES3G5M*, further suggests that Adaptive GRU is particularly beneficial for short and sparse interaction sequences, where fine-grained local patterns tend to carry more informative signals. Furthermore, replacing Adaptive GRU with a standard GRU by removing only the length-aware gate (*w/o Length-aware Gate*) still underperforms the full model, especially on the *Junyi* dataset where short interaction sequences are more prevalent. This highlights the role of sequence-level conditioning in improving sensitivity to local patterns under limited data conditions.

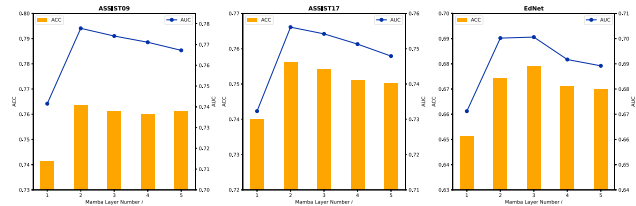
Disabling the student ability weighting (*w/o Student Weighting*) by setting all $w_i = 1$ also results in consistent performance degradation, particularly on XES3G5M and POJ. This confirms that incorporating individual student proficiency improves the estimation of exercise difficulty and contributes meaningfully to overall predictive performance.

The Parallel-RNN variant (*Parallel-RNN*) outperforms the full model on the Junyi dataset, likely due to the dataset's relatively short and randomly sampled sequences. In such cases, RNNs may outperform state space models in capturing short-term dependencies. Replacing the dynamic mix layer with simple feature concatenation (*Concat Fusion*) also leads to weaker results, indicating the importance of adaptive feature fusion. Finally, the Single Mamba variant (*Single Mamba*) underperforms the dual-path architecture. This degradation suggests that a single Mamba module struggles to disentangle heterogeneous dynamics when response and difficulty signals are jointly modeled within the same state transition, leading to representational interference. This observation further validates the necessity of the dual-stream design.

Furthermore, we compare three fusion strategies—element-wise addition, feed-forward transformation over concatenated features (Concat+FFN), and gated fusion—as described in Section IV-E, with results summarized in Table 9. Gated fusion achieves the best performance across all datasets by dynamically adjusting the contribution of behavioral and difficulty representations at each timestep, allowing the model to better capture the varying importance of different input modalities. In contrast, addition treats both branches equally without adaptation, while Concat+FFN introduces nonlinearity but lacks contextual flexibility. These

TABLE 9. The comparison of different fusion strategies in MCKT.

Fusion Strategy	XES3G5M		Junyi		POJ	
	AUC	ACC	AUC	ACC	AUC	ACC
Addition	0.8340	0.8337	0.8622	0.7969	0.7445	0.7358
Concat+FFN	0.8380	0.8352	0.8653	0.7990	0.7471	0.7384
Gated Fusion (Ours)	0.8451	0.8385	0.8688	0.7991	0.7524	0.7412

**FIGURE 4.** The sensitivity analysis of the number of Mamba layers in the MKT model.

findings highlight the effectiveness of gated fusion in enhancing representational capacity and generalization.

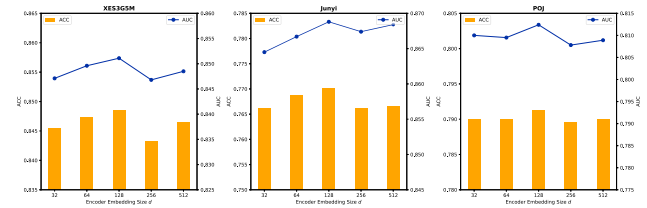
F. SENSITIVITY ANALYSIS (RQ4)

To examine the sensitivity of our model to critical hyperparameters, we conduct a set of experiments evaluating how variations in the number of Mamba layers in MKT and the embedding size in MCKT affect performance across multiple datasets. This analysis aims to investigate the robustness of MCKT under different architectural configurations and to identify optimal hyperparameter settings for knowledge tracing tasks. By systematically adjusting these components, we seek to better understand the influence of design choices on both predictive performance and model scalability.

Fig. 4 presents the effect of different Mamba layer numbers in MKT. The results indicate that using only a single Mamba layer without residual connections leads to a noticeable performance drop, suggesting that deeper architectures enhance the model's ability to capture complex sequential dependencies. However, adding excessive layers does not yield further gains, implying diminishing returns. Similarly, Fig. 5 examines the effect of embedding size in MCKT. While performance remains relatively stable across different embedding sizes, the results show slight variations, indicating that excessively small embedding sizes may limit feature representation capacity, whereas very large embeddings introduce redundancy without substantial benefits. These findings confirm that MCKT maintains robustness across a range of hyperparameter choices while highlighting optimal configurations for practical deployment.

To further assess the robustness of MCKT, we conduct a sensitivity analysis on the definition of the student ability weight w_i . Table 10 reports the model's performance on three datasets under different weighting strategies, including:

- **w/o Student Weighting:** Student ability is ignored, and all w_i values are set to 1.

**FIGURE 5.** The sensitivity analysis of the encoder embedding size in the MCKT model.**TABLE 10.** The sensitivity analysis of MCKT under alternative student ability weight w_i .

Weighting Variant	XES3G5M		Junyi		POJ	
	AUC	ACC	AUC	ACC	AUC	ACC
w/o w_i	0.8437	0.8380	0.8523	0.7953	0.7487	0.7389
Binary(0.7)	0.8421	0.8369	0.8516	0.7941	0.7456	0.7367
$\log(1 + w_i)$	0.8443	0.8372	0.8685	0.7986	0.7526	0.7412
Individual Proficiency	0.8451	0.8385	0.8688	0.7991	0.7524	0.7411

- **Binary Classification:** Students are assigned $w_i = 1$ if their overall accuracy exceeds 0.7, and $w_i = 0$ otherwise.
- **Logarithmic Scaling:** We compress the original proficiency scores using $w_i = \log(1 + w_i)$, which reduces variance while preserving ranking.

As shown in Table 10, removing the student ability weight (w/o w_i) leads to a consistent drop in both AUC and ACC across all three datasets, which echoes the results in our ablation study and validates the importance of individualized proficiency modeling. The *Binary* variant discretizes student ability using a fixed threshold of 0.7. Although simple, this coarse granularity fails to capture subtle differences between students, resulting in degraded performance. The *Log-scale*($\log(1 + w_i)$) transformation compresses high values of w_i , effectively smoothing large ability disparities. This variant shows slight improvements on POJ, particularly when student ability distributions are skewed. Nevertheless, the original *Individual Proficiency* strategy remains the best-performing choice overall. These results confirm that while MCKT benefits from fine-grained ability estimation, it maintains stable performance under simplified or transformed variants, enhancing its robustness and practical applicability.

To further examine the robustness of the difficulty formulation in Eq. 15, we conduct a sensitivity analysis on the minimum number of responses required for estimating difficulty, as well as an alternative smoothing strategy. The original setting requires at least 50 responses for an exercise before estimating its difficulty.

We first vary the response threshold $T \in \{10, 25, 50, 100\}$ and report the resulting predictive performance. As an alternative to the hard fallback rule, we consider a smoothing strategy based on Laplace smoothing. Specifically, the smoothed difficulty is defined as follows:

$$\tilde{\beta}_j = 1 - \frac{\sum_i w_i |R_+^{i,j}| + \alpha}{\sum_i w_i |R^{i,j}| + 2\alpha},$$

We report results using Laplace smoothing with $\alpha = 1$, and additionally test a stronger smoothing setting with $\alpha = 5$ to examine robustness. Although the degree of sensitivity to the threshold varies slightly across datasets, the overall performance differences remain small; considering fairness and computational efficiency, we retain 50 as the default threshold in this work.

TABLE 11. The sensitivity analysis on difficulty threshold and smoothing strategy.

Variant	ASSIST09		ASSIST17		EdNet	
	AUC	ACC	AUC	ACC	AUC	ACC
Threshold = 10	0.7765	0.7623	0.7554	0.7155	0.6990	0.6730
Threshold = 25	0.7772	0.7630	0.7558	0.7159	0.6997	0.6737
Threshold = 50	0.7778	0.7634	0.7561	0.7161	0.7002	0.6741
Threshold = 100	0.7769	0.7627	0.7557	0.7158	0.6994	0.6734
Laplace smoothing ($\alpha = 1$)	0.7781	0.7636	0.7561	0.7160	0.7005	0.6743
Laplace smoothing ($\alpha = 5$)	0.7774	0.7631	0.7559	0.7160	0.7008	0.6744

G. MODEL VISUALIZATION

We assess the impact of the MCKT model on exercise embeddings by randomly selecting 500 exercises and dividing them into five difficulty levels according to their accuracy rates in the training set, with each group containing 100 exercises. We employed t-SNE to project the high-dimensional embeddings into a two-dimensional space, comparing the initial embeddings with those processed by MCKT. As shown in the left subfigure of Fig. 6, the initial embeddings exhibit a scattered distribution without clear clustering based on difficulty levels. In contrast, the right subfigure demonstrates that the embeddings processed by MCKT exhibit significantly improved structure, with exercises of similar difficulty being projected into closer regions.

Notably, exercises from the lowest difficulty group (level 1) and the highest difficulty group (level 5) are positioned farthest apart in the t-SNE space. This observation suggests that the MCKT model effectively captures the differences in exercise difficulty, mapping them appropriately in the embedding space. This structured separation indicates that MCKT successfully integrates both exercise textual features and student response behaviors, enabling a more interpretable and effective knowledge tracing model.

Moreover, the role of the length-aware update gate in the Adaptive GRU is further examined through a sequence-length-based performance comparison. Specifically, we compare the full design with a variant where the sequence-level context term (i.e., the average input vector $\bar{x}$) is removed from the update gate, reducing it to a standard GRU formulation. To better reveal the effect across different interaction histories, we extend the maximum sequence length to 200 (instead of the default 50), while filtering out sequences shorter than 3 interactions. As illustrated in Fig. 7,

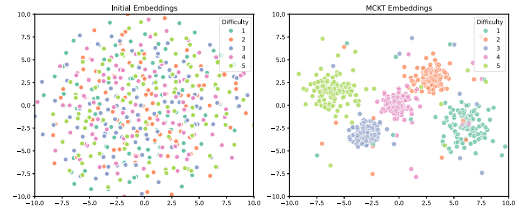


FIGURE 6. The visualization of exercise embeddings. The left figure shows the initial embeddings without clear difficulty-based clustering, while the right figure presents the MCKT-transformed embeddings, where similar difficulty levels are more clustered, and the hardest and easiest exercises are the most separated.

we group sequences into four ranges: 3–10, 11–50, 51–100, and 101–200.

The results reveal several patterns. In the shortest range 3–10, performance is relatively low across both variants due to insufficient historical signals, but the variant with the length-aware gate still shows a noticeable advantage, indicating its ability to extract meaningful patterns from sparse data. In the most common range (51–100), the performance gap is more stable and substantial, demonstrating the consistent benefit of adaptive gating in moderately long sequences. Interestingly, in the longest range (101–200), the variant without the length-aware gate slightly outperforms the gated version, possibly because the GRU gate may introduce redundancy when ample sequential information is already present. Nonetheless, the overall trend supports the gate's value, especially in short to moderate-length sequences, which are also the most commonly observed in real-world educational scenarios.

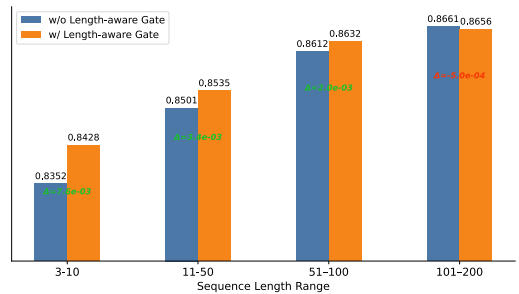


FIGURE 7. The effect of the length-aware gate across sequence length ranges on Junyi. The variant with the gate (orange bars) consistently outperforms the baseline (blue bars) in short to moderate-length sequences. The annotated performance gain (Δ) diminishes as sequence length increases.

To further support the interpretability claim, we conduct a lightweight faithfulness analysis based on the learned summary attention weights used in the final prediction stage. Specifically, for each test sequence, we rank the valid historical interactions according to these weights and then mask 20% of interactions using three strategies: masking the highest-weight interactions, masking the lowest-weight interactions, and masking randomly selected interactions.

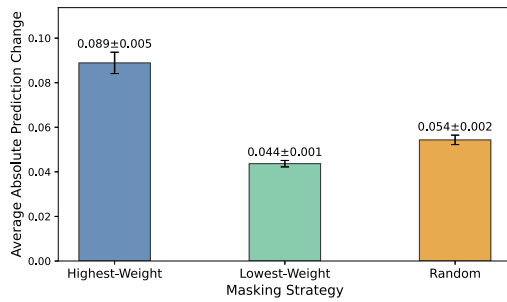


FIGURE 8. The faithfulness analysis of the learned summary attention weights. The x-axis shows three masking strategies: highest-weight masking, lowest-weight masking, and random masking. The y-axis reports the average absolute change between the original and masked predicted probabilities. Error bars denote the standard deviation across test batches.

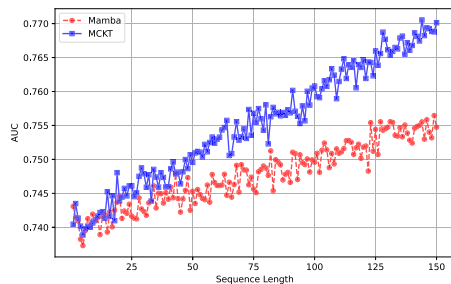


FIGURE 9. The AUC performance across different sequence lengths.

We then quantify the effect of masking by computing the mean absolute difference between the predicted probabilities from the original sequence and those from the masked sequence, averaged over all valid prediction positions. As shown in Fig. 8, Highest-Weight Masking yields the largest prediction change (0.089 ± 0.005), followed by Random Masking (0.054 ± 0.002), while Lowest-Weight Masking produces the smallest change (0.044 ± 0.001). This ordering suggests that the learned summary attention weights capture the relative importance of historical interactions in the final prediction stage. Therefore, the faithfulness analysis offers further evidence for the interpretability claim by showing that these weights are meaningfully related to the model's prediction process.

H. CASE STUDY

To investigate the impact of Adaptive GRU in MCKT, we conducted a case study comparing MCKT with and without Adaptive GRU. Fig. 9 illustrates the relationship between sequence length and prediction performance, providing insights into how Adaptive GRU influences knowledge tracing effectiveness across different interaction histories.

The results indicate that MCKT without Adaptive GRU struggles with short sequences (<20 exercises), exhibiting unstable predictions and a noticeable drop in accuracy. In contrast, MCKT with Adaptive GRU significantly improves performance for short sequences while maintaining comparable accuracy for longer sequences

(>50 exercises). Notably, the enhancement is most evident in students with limited interactions, where Adaptive GRU helps stabilize predictions and align them more closely with actual correctness. These findings highlight that Adaptive GRU plays a crucial role in mitigating the long-tail issue, improving robustness in short-sequence learning scenarios, and enhancing the overall effectiveness of MCKT.

VI. CONCLUSION AND PERSPECTIVES

This paper introduces MCKT, a content-aware knowledge tracing framework built upon a parallel Mamba architecture, designed to jointly model exercise semantics and student behavior for enhanced prediction accuracy. The model incorporates a content-aware embedding mechanism leveraging a pre-trained text encoder, introduces exercise difficulty as an explicit modeling signal inspired by the Rasch model, and employs a parallel-branch Mamba architecture with a dedicated mixing layer to fuse the two input streams. Furthermore, an Adaptive GRU module is introduced to address the long-tail user problem in sparse interaction scenarios. Extensive experiments demonstrate that MCKT consistently outperforms state-of-the-art knowledge tracing models, and its design also suggests broad applicability to other sequential learning tasks such as personalized recommendation and user behavior modeling. Looking ahead, we plan to enhance MCKT by incorporating large language models to automatically assess exercise difficulty levels, reducing reliance on manual annotations [67]. Additionally, federated learning will be explored to strengthen personalization and privacy protection, enabling decentralized training across institutions [68], [69]. Moreover, we aim to extend MCKT to a multimodal framework by incorporating visual information from exercises, particularly for subjects like mathematics and physics where visual reasoning plays a central role [70]. Another promising direction is to incorporate graph neural network (GNN) techniques into MCKT. Knowledge concepts and exercises naturally form graph-structured relationships, and GNNs could complement Mamba's sequential modeling by capturing topological dependencies among concepts. Recent advances in graph representation learning, such as multi-topology contrastive approaches [71] and theoretical insights into the stability and generalization of deep graph convolutional networks [72], offer promising foundations for such an extension. Furthermore, domain-specific pretraining of the content encoder will be explored to improve exercise representation quality. Notably, while the frozen encoder performs comparably to fine-tuning on the current datasets, which consist primarily of short and structured mathematical and programming exercises, for disciplines with richer and more complex textual content, such as liberal arts, domain-adaptive fine-tuning is likely to yield more substantial benefits. Meta-learned adaptation strategies for the Adaptive GRU will also be investigated to improve robustness in sparse interaction scenarios.

APPENDIX A SUPPLEMENTARY ANALYSIS: FINE-TUNING STRATEGIES FOR RoBERTa ENCODER

A. OVERVIEW AND MOTIVATION

In our main experiments, the Content Embedder module of MCKT utilizes a **frozen** RoBERTa-base model to encode question texts. This choice aims to reduce training overhead while leveraging the representational power of pre-trained language models. To further investigate the potential benefits of adapting the encoder to the educational domain, we conduct a supplementary analysis by exploring two representative fine-tuning strategies:

- **Domain-Adaptive Pretraining (DAPT):** Perform masked language modeling (MLM) on in-domain educational texts extracted from the exercise content in our training set.
- **End-to-End Fine-tuning (E2E):** Integrate the RoBERTa encoder into the MCKT architecture as a trainable module and update its parameters during downstream prediction training.

B. METHODOLOGICAL DETAILS

1) DAPT

Let $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ denote the textual content of exercises in the training set. We continue pretraining the RoBERTa-base model on $\mathcal{T}$ using the masked language modeling (MLM) objective:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i=1}^n \log p_{\theta}(w_i | \hat{w}_{\setminus i}), \quad (46)$$

where w_i is a masked token and $\hat{w}_{\setminus i}$ is the surrounding context. Once the adaptation is complete, the encoder is frozen and used to regenerate static semantic embeddings for all questions, as in the MCKT model setup.

2) E2E

In this setting, the RoBERTa encoder $E(\cdot)$ becomes a differentiable module within MCKT. The full prediction loss (e.g., binary cross-entropy) is backpropagated through both MCKT and $E(\cdot)$:

$$\mathcal{L}_{\text{BCE}} = - \sum_t [r_t \log \hat{r}_t + (1 - r_t) \log(1 - \hat{r}_t)], \quad (47)$$

where r_t and $\hat{r}_t$ are the ground-truth and predicted responses. We adopt a smaller learning rate for RoBERTa parameters to ensure stable optimization.

C. EXPERIMENTAL SETUP

To construct an in-domain educational corpus for DAPT, we collect question texts from the training splits of three content-rich educational datasets spanning mathematical and programming domains. After filtering and preprocessing, we obtain a comprehensive corpus consisting of exactly **687,067** entries, totaling approximately **110 MB** of raw text.

For the DAPT strategy, we further pretrain the RoBERTa-base model on this educational corpus using the masked language modeling (MLM) objective for 10 epochs, with a learning rate of $5e-5$ and a batch size of 64. The resulting adapted encoder is then used to generate fixed question-level semantic embeddings for each exercise, which are subsequently passed into the downstream MCKT model.

For the E2E strategy, we initialize the same RoBERTa-base model without DAPT and incorporate it as a trainable module within the MCKT architecture. The RoBERTa parameters are updated jointly with the rest of the model during task-specific training, using a smaller learning rate (e.g., $1e-4$) to ensure stable convergence.

All fine-tuning experiments are conducted separately on each dataset to evaluate the impact of different adaptation strategies across diverse educational contexts.

D. RESULTS AND ANALYSIS

We evaluate both fine-tuning strategies across three educational datasets and compare them to the frozen RoBERTa baseline within the MCKT framework. Table 12 summarizes the results in terms of AUC and ACC on XES3G5M, Junyi, and POJ.

TABLE 12. The performance comparison of RoBERTa fine-tuning strategies on three datasets.

Strategy	XES3G5M		Junyi		POJ	
	AUC	ACC	AUC	ACC	AUC	ACC
Frozen RoBERTa	84.51	83.85	86.88	79.91	75.24	74.12
DAPT RoBERTa	84.32	83.62	86.64	79.53	74.98	73.86
E2E Fine-tuning	84.48	83.79	86.91	79.89	75.23	74.10

As reported in Table 12, both fine-tuning strategies yield performance comparable to the frozen RoBERTa baseline, with limited gains. Notably, the DAPT strategy underperforms slightly relative to the frozen model across all three datasets, likely because the exercise content corpus mainly includes short and repetitive sentences, many of which are simple question-like expressions or answer choices. This lack of linguistic diversity limits its ability to further improve the encoder's performance. The end-to-end fine-tuning approach achieves a marginal improvement in AUC (0.03%) only on the Junyi dataset, while exhibiting no substantial advantage—or even slight degradation—on XES3G5M and POJ. This outcome may reflect the susceptibility of full-model tuning to overfitting, particularly under data scarcity or when task-specific supervision signals are weak. These observations indicate that the improvements from fine-tuning are minimal and may not warrant the additional computational cost incurred by full fine-tuning in practical deployments.

E. CONCLUSION

Overall, the performance of the frozen encoder highlights RoBERTa's inherent ability to generate content-aware

semantic features that generalize well to knowledge tracing tasks, even without task-specific adaptation. While fine-tuning brings slight improvements, it also introduces significant resource demands and optimization challenges. Given that one of MCKT's core objectives is to balance predictive performance with efficiency—leveraging lightweight architectures like Mamba—we retain the frozen encoder as the default setting. Exploring more advanced domain-adaptive techniques remains a promising direction for future work.

REFERENCES

- [1] A. Sangrà, D. Vlachopoulos, and N. Cabrera, "Building an inclusive definition of e-learning: An approach to the conceptual framework," *Int. Rev. Res. Open Distrib. Learn.*, vol. 13, no. 2, pp. 145–159, Apr. 2012.
- [2] W. Al-Rahmi, A. Aldraiweesh, N. Yahaya, Y. Bin Kamin, and A. M. Zeki, "Massive open online courses (MOOCs): Data on higher education," *Data Brief*, vol. 22, pp. 118–125, Feb. 2019.
- [3] H. Burns, C. A. Luckhardt, J. W. Parlett, and C. L. Redfield, *Intelligent Tutoring Systems: Evolutions in Design*. Hove, U.K.: Psychology Press, 2014.
- [4] A. Khalid, K. Lundqvist, and A. Yates, "A literature review of implemented recommendation techniques used in massive open online courses," *Expert Syst. Appl.*, vol. 187, Jan. 2022, Art. no. 115926.
- [5] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Model. User-Adapted Interact.*, vol. 4, no. 4, pp. 253–278, 1995.
- [6] S. Shen, Q. Liu, Z. Huang, Y. Zheng, M. Yin, M. Wang, and E. Chen, "A survey of knowledge tracing: Models, variants, and applications," *IEEE Trans. Learn. Technol.*, vol. 17, pp. 1858–1879, 2024.
- [7] Y. Yang, J. Shen, Y. Qu, Y. Liu, K. Wang, Y. Zhu, W. Zhang, and Y. Yu, "GKKT: A graph-based interaction model for knowledge tracing," in *Proc. Mach. Learn. Knowl. Discovery Databases, Eur. Conf.*, Ghent, Belgium, Sep. 2020, pp. 299–315.
- [8] M. Yudelson, K. R. Koedinger, and G. J. Gordon, "Individualized Bayesian knowledge tracing models," in *Proc. Artif. Intell. Educ., 16th Int. Conf.*, Memphis, TN, USA, 2013, pp. 171–180.
- [9] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, "Deep knowledge tracing," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 505–513.
- [10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [11] J. Zhang, X. Shi, I. King, and D.-Y. Yeung, "Dynamic key-value memory networks for knowledge tracing," in *Proc. 26th Int. Conf. World Wide Web*, Apr. 2017, pp. 765–774.
- [12] S. Pandey and G. Karypis, "A self-attentive model for knowledge tracing," in *Proc. 12th Int. Conf. Educ. Data Mining*, 2019, pp. 384–389.
- [13] Y. Choi, L. Younghan, C. Junghyun, J. Baek, B. Kim, Y. Cha, D. Shin, C. Bae, and J. Heo, "Towards an appropriate query, key, and value computation for knowledge tracing," in *Proc. 7th ACM Conf. Learning Scale*, 2020, pp. 341–344.
- [14] C. Cui, Y. Yao, C. Zhang, H. Ma, Y. Ma, Z. Ren, C. Zhang, and J. Ko, "DGEKT: A dual graph ensemble learning method for knowledge tracing," *ACM Trans. Inf. Syst.*, vol. 42, no. 3, pp. 1–24, May 2024.
- [15] S.-H. Noh, "Analysis of gradient vanishing of RNNs and performance comparison," *Information*, vol. 12, no. 11, p. 442, Oct. 2021.
- [16] C. Zhu, W. Ping, C. Xiao, M. Shoenybi, T. Goldstein, A. Anandkumar, and B. Catanzaro, "Long-short transformer: Efficient transformers for language and vision," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 17723–17736.
- [17] R. K. Hambleton, H. Swaminathan, and H. J. Rogers, *Fundamentals of Item Response Theory*, vol. 2. Newbury Park, CA, USA: Sage, 1991.
- [18] G. Rasch, *Probabilistic Models for Some Intelligence and Attainment Tests*. Stockholm, Sweden: ERIC, 1993.
- [19] J. Chen, S. Huang, Q. Liu, Z. Liu, W. Luo, and J. Tang, "PyKT: A Python library to benchmark deep learning based knowledge tracing models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 18542–18555.
- [20] D. Liu, L. Guo, X. r. zhang, and Y. Li, "ETVKT: Enhanced training vector for knowledge tracing," in *Proc. Int. Conf. Intell. Comput.*, 2024, pp. 474–481.
- [21] Z. Wang, J. Zhou, Q. Chen, M. Zhang, B. Jiang, A. Zhou, Q. Bai, and L. He, "LLM-KT: Aligning large language models with knowledge tracing using a plug-and-play instruction," 2025, *arXiv:2502.02945*.
- [22] J.-J. Vie and H. Kashima, "Knowledge tracing machines: Factorization machines for knowledge tracing," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 750–757.
- [23] G. Liu, H. Zhan, and J.-J. Kim, "Question difficulty consistent knowledge tracing," in *Proc. ACM Web Conf.*, May 2024, pp. 4239–4248.
- [24] S. Shen, Z. Huang, Q. Liu, Y. Su, S. Wang, and E. Chen, "Assessing student's dynamic knowledge state by exploring the question difficulty effect," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2022, pp. 427–437.
- [25] K. Cheng, L. Peng, P. Wang, J. Ye, L. Sun, and B. Du, "DyGKT: Dynamic graph learning for knowledge tracing," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2024, pp. 409–420.
- [26] H. Zhang, Z. Liu, C. Shang, D. Li, and Y. Jiang, "A question-centric multi-experts contrastive learning framework for improving the accuracy and interpretability of deep sequential knowledge tracing models," *ACM Trans. Knowl. Discovery From Data*, vol. 19, no. 2, pp. 1–25, Feb. 2025.
- [27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Adv. Neural Inf. Process. Syst.*, vol. 30, pp. 5998–6008, 2017.
- [28] G. Abdelrahman and Q. Wang, "Deep graph memory networks for forgetting-robust knowledge tracing," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 8, pp. 7844–7855, Aug. 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/9891841>
- [29] T. Long, Y. Liu, J. Shen, W. Zhang, and Y. Yu, "Tracing knowledge state with individual cognition and acquisition estimation," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2021, pp. 173–182.
- [30] Y. Yin, L. Dai, Z. Huang, S. Shen, F. Wang, Q. Liu, E. Chen, and X. Li, "Tracing knowledge instead of patterns: Stable knowledge tracing with diagnostic transformer," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 855–864.
- [31] Z. Liu, Q. Liu, J. Chen, S. Huang, and W. Luo, "SimpleKT: A simple but tough-to-beat baseline for knowledge tracing," 2023, *arXiv:2302.06881*.
- [32] Q. Guan, X. Duan, K. Bian, G. Chen, J. Huang, Z. Gong, and L. Fang, "KVFKT: A new horizon in knowledge tracing with attention-based embedding and forgetting curve integration," in *Proc. 31st Int. Conf. Comput. Linguistics*, 2025, pp. 4399–4409.
- [33] Y. Pu, F. Liu, R. Shi, H. Yuan, R. Chen, T. Peng, and W. Wu, "ELAKT: Enhancing locality for attentive knowledge tracing," *ACM Trans. Inf. Syst.*, vol. 42, no. 4, pp. 1–27, Jul. 2024.
- [34] X. Li, Y. Bai, T. Guo, Z. Liu, Y. Huang, X. Zhao, F. Xia, W. Luo, and J. Weng, "Enhancing length generalization for attention based knowledge tracing models with linear biases," in *Proc. 33rd Int. Joint Conf. Artificial Intell. (IJCAI)*, 2024, pp. 5918–5926.
- [35] X. Li, Y. Bai, T. Guo, Y. Zheng, M. Hou, B. Zhan, Y. Huang, Z. Liu, B. Gao, and W. Luo, "Extending context window of attention based knowledge tracing models via length extrapolation," in *Proc. ECAI*, 2024, pp. 1479–1486.
- [36] A. Ghosh, N. Heffernan, and A. S. Lan, "Context-aware attentive knowledge tracing," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 2330–2339.
- [37] Y. Xiao, R. Xiao, N. Huang, Y. Hu, H. Li, and B. Sun, "Knowledge tracing based on multi-feature fusion," *Neural Comput. Appl.*, vol. 35, no. 2, pp. 1819–1833, Jan. 2023.
- [38] Y. Su, Q. Liu, Q. Liu, Z. Huang, Y. Yin, E. Chen, C. Ding, S. Wei, and G. Hu, "Exercise-enhanced sequential modeling for student performance prediction," in *Proc. 32nd AAAI Conf. Artif. Intell. 13th Innov. Appl. Artif. Intell. Conf. 8th AAAI Symp. Educ. Adv. Artif. Intell.*, vol. 32, 2018, pp. 2435–2443.
- [39] Q. Liu, Z. Huang, Y. Yin, E. Chen, H. Xiong, Y. Su, and G. Hu, "EKT: exercise-aware knowledge tracing for student performance prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 1, pp. 100–115, Jan. 2021.
- [40] S. Pandey and J. Srivastava, "RKT: Relation-aware self-attention for knowledge tracing," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2020, pp. 1205–1214.
- [41] S. Cheng, Q. Liu, E. Chen, K. Zhang, Z. Huang, Y. Yin, X. Huang, and Y. Su, "AdaptKT: A domain adaptable method for knowledge tracing," in *Proc. 15th ACM Int. Conf. Web Search Data Mining*, Feb. 2022, pp. 123–131.
- [42] N. Liu, Z. Wang, R. Baraniuk, and A. Lan, "Open-ended knowledge tracing for computer science education," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2022, pp. 3849–3862.

- [43] L. Li and Z. Wang, "Calibrated Q-matrix-enhanced deep knowledge tracing with relational attention mechanism," *Appl. Sci.*, vol. 13, no. 4, p. 2541, Feb. 2023.
- [44] L. Fu, H. Guan, K. Du, J. Lin, W. Xia, W. Zhang, R. Tang, Y. Wang, and Y. Yu, "SINKT: A structure-aware inductive knowledge tracing model with large language model," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2024, pp. 632–642.
- [45] W. Gao, Q. Liu, L. Yue, F. Yao, R. Lv, Z. Zhang, H. Wang, and Z. Huang, "Agent4Edu: Generating learner response data by generative agents for intelligent education systems," 2025, *arXiv:2501.10332*.
- [46] Y. Guo, S. Shen, Q. Liu, Z. Huang, L. Zhu, Y. Su, and E. Chen, "Mitigating cold-start problems in knowledge tracing with large language models: An attribute-aware approach," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2024, pp. 727–736.
- [47] Q. Fournier, G. M. Caron, and D. Aloise, "A practical survey on faster and lighter transformers," *ACM Comput. Surv.*, vol. 55, no. 14s, pp. 1–40, 2023.
- [48] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," 2021, *arXiv:2111.00396*.
- [49] J. T. H. Smith, A. Warrington, and S. W. Linderman, "Simplified state space layers for sequence modeling," 2022, *arXiv:2208.04933*.
- [50] D. Y. Fu, T. Dao, K. K. Saab, A. W. Thomas, A. Rudra, and C. Ré, "Hungry hungry hippos: Towards language modeling with state space models," 2022, *arXiv:2212.14052*.
- [51] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.
- [52] C. Liu, J. Lin, J. Wang, H. Liu, and J. Caverlee, "Mamba4Rec: Towards efficient sequential recommendation with selective state space models," 2024, *arXiv:2403.03900*.
- [53] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, "Vision mamba: Efficient visual representation learning with bidirectional state space model," 2024, *arXiv:2401.09417*.
- [54] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.
- [55] X. Wang, S. Wang, Y. Ding, Y. Li, W. Wu, R. Yao, W. Kong, J. Huang, S. Li, H. Yang, Z. Wang, B. Jiang, C. Li, Y. Wang, Y. Tian, and J. Tang, "State space model for new-generation network alternative to transformers: A survey," 2024, *arXiv:2404.09516*.
- [56] Y. Cao and W. Zhang, "Mamba4KT: An efficient and effective mamba-based knowledge tracing model," 2024, *arXiv:2405.16542*.
- [57] J. Sun, F. Yu, Q. Wan, Q. Li, S. Liu, and X. Shen, "Interpretable knowledge tracing with multiscale state representation," in *Proc. ACM Web Conf.*, May 2024, pp. 3265–3276.
- [58] Z. Li, J. Yang, J. Wang, L. Shi, J. Feng, and S. Stein, "LBKT: A LSTM bert-based knowledge tracing model for long-sequence data," in *Proc. Int. Conf. Intell. Tutoring Syst.* Cham, Switzerland: Springer, 2024, pp. 174–184.
- [59] Z. Liu, Q. Liu, Y. Wang, W. Wang, P. Jia, M. Wang, Z. Liu, Y. Chang, and X. Zhao, "SIGMA: Selective gated mamba for sequential recommendation," 2024, *arXiv:2408.11451*.
- [60] A. Gu, I. Johnson, K. Goel, K. Saab, T. Dao, A. Rudra, and C. Ré, "Combining recurrent, convolutional, and continuous-time models with linear state space layers," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 572–585.
- [61] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "HiPPO: Recurrent memory with optimal polynomial projections," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1474–1487.
- [62] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
- [63] M. Feng, N. T. Heffernan, and K. R. Koedinger, "Addressing the assessment challenge with an online system that tutors as it assesses," in *User Modeling and User-Adapted Interaction*, Springer, Dordrecht, Netherlands, vol. 19, 2009, pp. 243–266.
- [64] J. Choi, Y. Lee, D. Shin, J. Cho, S. Park, S. Lee, J. Baek, C. Bae, B. Kim, and J. Heo, "EdNet: A large-scale hierarchical dataset in education," in *Proc. 21st Int. Conf. Artif. Intell. Educ.*, Ifrane, Morocco, 2020, pp. 69–73.
- [65] J. Chen, T. Guo, S. Huang, Q. Liu, Z. Liu, W. Luo, J. Tang, J. Weng, and X. Zhao, "XES3G5M: A knowledge tracing benchmark dataset with auxiliary information," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 32958–32970.
- [66] H.-S. Chang, H.-J. Hsu, and K. Chen, "Modeling exercise relationships in E-learning: A unified approach," in *Proc. EDM*, 2015, pp. 532–535.
- [67] J. Cen, J. Lin, W. Zhong, D. Zhou, J. Chen, A. Yang, and Y. Zhou, "LLM-driven effective knowledge tracing by integrating dual-channel difficulty," 2025, *arXiv:2502.19915*.
- [68] C. Chen, J. Liu, H. Tan, X. Li, K. I.-K. Wang, P. Li, K. Sakurai, and D. Dou, "Trustworthy federated learning: Privacy, security, and beyond," *Knowl. Inf. Syst.*, vol. 67, no. 3, pp. 2321–2356, Mar. 2025.
- [69] Q. Liu, J. Wu, Z. Huang, H. Wang, Y. Ning, M. Chen, E. Chen, J. Yi, and B. Zhou, "Federated user modeling from hierarchical information," *ACM Trans. Inf. Syst.*, vol. 41, no. 2, pp. 1–33, Apr. 2023.
- [70] X. Ding, T. Han, Y. Fang, and E. Larson, "An approach for combining multimodal fusion and neural architecture search applied to knowledge tracing," *Appl. Intell.*, vol. 53, no. 9, pp. 11092–11103, May 2023.
- [71] Y. Xie, J. Jia, C. Wen, D. Li, and M. Li, "Multi-topology contrastive graph representation learning," *Sci. China Inf. Sci.*, vol. 69, no. 2, Feb. 2026, Art. no. 122102.
- [72] G. Yang, M. Li, H. Feng, and X. Zhuang, "Deeper insights into deep graph convolutional networks: Stability and generalization," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 48, no. 2, pp. 1707–1719, Feb. 2026.



YUMO YAO is currently pursuing the Ph.D. degree with the Graduate School of Comprehensive Human Sciences, University of Tsukuba, Japan. His research interests include educational data mining, knowledge tracing, federated learning, and machine learning. He has also been involved in organizing international conferences, such as the 2024 SIGIR-AP.



HAI-TAO YU received the Ph.D. degree from the Faculty of Engineering, University of Tokushima, Japan, in 2013. He is currently a tenured Associate Professor with the University of Tsukuba. His current research interests include trustworthy information retrieval, retrieval-augmented LLM, and commonsense reasoning. By now, he has been actively involved in co-organizing multiple conferences, such as the 2024 SIGIR-AP and the 2024 Australasian Database Conference; evaluation tasks, such as NTCIR Temporal Information Access and NTCIR Actionable Knowledge Graph; and workshops, such as When Creative AI Meets Conversational AI and Refined and Reliable Retrieval Augmented Generation.

• • •